International Islamic University Chittagong

IIUC_StrawHats

Afrin Hasan Safin, Jillur Rahman, Md. Arif

ICPC Asia Dhaka Regional Contest

December 20, 2025

Contents

Contest (1)

run.sh

8 lines

```
run() {
    code="$1"
    g++ "${code}.cpp" -o "$code" -std=c++20 -g -DDeBuG
        \
        -Wall -Wshadow -fsanitize=address,undefined \
        && "./$code"
}

source ~/.bashrc
```

s.sh

7 lines

```
# run by   bash s.sh

for((i = 1; i<=100 ; ++i)); do
    echo $i
    ./gen $i > int
    diff -w <(./a< int) <(./brute < int) || break
done
```

tem.cpp

"bits/stdc++.h" 717f64, 30 lines

```
using namespace std;
#ifndef DeBuG
  #define dbg(...)
#endif

typedef long double ld;
typedef vector<int> vi;

#define sz(x) (int)(x).size()
#define all(x) begin(x), end(x)
#define ff first
#define ss second
#define pb push_back
#define nl '\n'
#define sp " "
#define fora(cn) for(auto &x : (cn))
#define rep(i,a,b) for(int i=(a);i<(b);++i)
#define forcin(p) for (auto &x : p) cin >> x

template<class T>using V=vector<T>;
using ll=long long; using pll = pair<ll,ll>;
using pii=pair<int,int>;using vi=vector<int> ;
using vl = vector<ll>  ;

int main() {
  ios_base::sync_with_stdio(0);
  cin.tie(0); cout.tie(0);

return 0 ;
}
```

stdc++.h

90f4a7, 29 lines

```
#include <bits/stdc++.h>
using namespace std;
```

```
#define TT template <typename T

TT,typename=void> struct cerrok:false_type {};
TT> struct cerrok <T, void_t<decltype(cerr << declval<T
    >() )>> : true_type {};

TT> constexpr void p1 (const T &x);
TT, typename V> void p1(const pair<T, V> &x) {
  cerr << "{"; p1(x.first); cerr << ", ";
  p1(x.second); cerr << "}";
}
TT> constexpr void p1 (const T &x) {
  if constexpr (cerrok<T>::value) cerr << x;
  else { int f = 0; cerr << '{';
    for (auto &i: x)
      cerr << (f++ ? ", " : ""), p1(i);
    cerr << "}";
} }
void p2() { cerr << "]\n"; }
TT, typename... V> void p2(T t, V... v) {
  p1(t);
  if (sizeof...(v)) cerr << ", ";
  p2(v...);
}

#ifdef DeBuG
#define dbg(x...) {cerr << "\t\e[93m"<<__func__<<":"<<
    __LINE__<<" [" << #x << "] = ["; p2(x); cerr << "\e
    [0m";}
#endif
```

Data structures (2)

2.0.1 PBDS

PBDS.cpp

01f254, 24 lines

```
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
using namespace std;
using namespace __gnu_pbds;
template < typename PB >
  using pbds = tree < PB, null_type, less < PB > ,
      rb_tree_tag, tree_order_statistics_node_update >
      ;
// or

typedef tree<PB,null_type,less<PB>,rb_tree_tag,
    tree_order_statistics_node_update> pbds;

example =>
 * - less<data_type> --> Increasingly sorted set
 * - less_equal<data_type> --> Increasingly  multiset
 * - greater<data_type> --> Decreasingly sorted set
 * - greated_equal<data_type> --> Decreasingly sorted
     multiset
 order_of_key(k): Number of items strictly smaller than
      k
 name.order_of_key(100);
find_by_order(k): returns an iterator , K-th element in
     a set ( Counting from zero)
 *name.find_by_order(5);

  Path of the File that needs to be renamed in case of
      error in including PBDS header files: C\Program
      Files\mingw-w64\x86_64-8.1.0-posix-seh-rt_v6-rev0
      \mingw64\lib\gcc\x86_64-w64-mingw32\8.1.0\include
      \c++\ext\pb_ds\detail\resize_policy
 // rename a file which has .hpp0000...lots of numbers
     to .hpp
```

2.0.2 Union Find

UnionFind.h

Description: Disjoint-set data structure.
Time: $\mathcal{O}(\alpha(N))$

7aa27c, 14 lines

```
struct UF {
  vi e;
  UF(int n) : e(n, -1) {}
  bool sameSet(int a, int b) { return find(a) == find(b
      ); }
  int size(int x) { return -e[find(x)]; }
  int find(int x) { return e[x] < 0 ? x : e[x] = find(e
      [x]); }
  bool join(int a, int b) {
    a = find(a), b = find(b);
    if (a == b) return false;
    if (e[a] > e[b]) swap(a, b);
    e[a] += e[b]; e[b] = a;
    return true;
  }
};
```

2.0.3 Union Find Rollback

UnionFindRollback.h

Description: Disjoint-set data structure with undo. If undo is not needed, skip st, time() and rollback().
Usage: `int t = uf.time(); ...; uf.rollback(t);`
Time: $\mathcal{O}(\log(N))$

de4ad0, 21 lines

```
struct RollbackUF {
  vi e; vector<pii> st;
  RollbackUF(int n) : e(n, -1) {}
  int size(int x) { return -e[find(x)]; }
  int find(int x) { return e[x] < 0 ? x : find(e[x]); }
  int time() { return sz(st); }
  void rollback(int t) {
    for (int i = time(); i --> t;)
      e[st[i].first] = st[i].second;
    st.resize(t);
  }
  bool join(int a, int b) {
    a = find(a), b = find(b);
    if (a == b) return false;
    if (e[a] > e[b]) swap(a, b);
    st.push_back({a, e[a]});
    st.push_back({b, e[b]});
    e[a] += e[b]; e[b] = a;
    return true;
  }
};
```

2.0.4 Fenwick Tree

FenwickTree.h

Description: `update(i,x):  a[i] += x;`
`query(i):  sum in [0, i);`
`lower_bound(sum):  min pos st sum of [0, pos] >= sum,` returns n if all < sum, or -1 if empty sum.
Time: Both operations are $\mathcal{O}(\log N)$.

003967, 16 lines

```
struct FT {
  int n; V<ll> s;
  FT(int _n) : n(_n), s(_n) {}
  void update(int i, ll x) {
    for (; i < n; i |= i + 1) s[i] += x; }
  ll query(int i, ll r = 0) {
    for (; i > 0; i &= i - 1) r += s[i-1]; return r; }
  int lower_bound(ll sum) {
    if (sum <= 0) return -1; int pos = 0;
    for (int pw = 1 << (31 - __builtin_clz(n)) ; pw; pw
        >>= 1){
      if (pos+pw <= n && s[pos + pw-1] < sum)
        pos += pw, sum -= s[pos-1];
    }
    return pos;
  }
};
```

2.0.5 Fenwick Tree (Range)

FenwickTreeRange.h

Description: Range add Range sum with FT.
Time: Both operations are $\mathcal{O}(\log N)$.

8fc549, 11 lines

```
FT f1(n), f2(n);
// a[l...r] += v; 0 <= l <= r < n
auto upd = [&](int l, int r, ll v) {
  f1.update(l, v), f1.update(r + 1, -v);
  f2.update(l, v*(l-1)), f2.update(r+1, -v*r);
}; // a[l] + ... + a[r]; 0 <= l <= r < n
auto sum = [&](int l, int r) { ++r;
  ll sub = f1.query(l) * (l-1) - f2.query(l);
  ll add = f1.query(r) * (r-1) - f2.query(r);
  return add - sub;
};
```

2.0.6 2D BIT

2Dbit.h

80b779, 20 lines

```
template <class T=int >
struct FT2D {
  V<V<T>> x;
  FT2D(int n, int m) : x(n, V<T>(m)) { }
```

```
  void add(int k1, int k2, int a) { // x[k1][k2] += a
    for (; k1 < sz(x) ; k1 |= k1+1)
      for (int k=k2; k < sz(x[k1]); k |= k+1) x[k1][k]
          += a;
  }
  T sum(int k1, int k2 ,T s=0 ) { // return  x[0][0] to
       x[k1][k2] sum

    for (; k1 >= 0; k1 = (k1&(k1+1))-1)
      for (int k=k2; k >= 0; k = (k&(k+1))-1) s += x[k1
          ][k];
    return s;
  }
  // sum of rectangle [(x1, y1), (x2, y2)] 0 base
   T rsum(int x1, int y1, int x2, int y2) {
        return sum(x2, y2) - (x1 ? sum(x1 - 1, y2) : 0)
             - (y1 ? sum(x2, y1 - 1) : 0)
             + (x1 && y1 ? sum(x1 - 1, y1 - 1) : 0);
    }
};
```

2.0.7 2D Prefix Sum

presum2d.h

b5b7f7, 17 lines

```
V<vl>  PrefixSums2D(V<vl> &a) {
  // ...1 base
    int  n = sz(a) ,m = sz(a[0]) ;
    V<vl> pre(n + 1, vl(m + 1, 0));
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < m; j++) {
            pre[i + 1][j + 1] = pre[i][j + 1] + pre[i +
                1][j] - pre[i][j] + a[i][j];
        }
    }
    return pre;
}
ll get(int rx,int ry,int lx,int ly,V<vl> &pre){
  // ... 1 base l to r inclusive
  lx--,--ly;
  return pre[rx][ry]+pre[lx][ly]-pre[rx][ly]-pre[lx][ry
      ];

}
```

2.0.8 RMQ

RMQ.h

Description: Range Minimum Queries on an array. Returns min(V[a], V[a + 1], ... V[b - 1]) in constant time.
Usage: `RMQ rmq(values);`
`rmq.query(inclusive, exclusive);`
Time: $\mathcal{O}(|V| \log |V| + Q)$

7d2211, 15 lines

```
template<class T>
struct RMQ {
  V<V<T>> jmp;
  RMQ(const V<T>& V) : jmp(1, V) {
    for (int pw = 1, k = 1; pw * 2 <= sz(V); pw *= 2,
        ++k) {
      jmp.emplace_back(sz(V) - pw * 2 + 1);
      rep(j,0,sz(jmp[k]))
        jmp[k][j] = min(jmp[k - 1][j], jmp[k - 1][j +
            pw]);
    }
  }
  T query(int a, int b) {
    assert(a < b); // or return inf if a == b
    int dep = 31 - __builtin_clz(b - a);
    return min(jmp[dep][a], jmp[dep][b - (1 << dep)]);
} };
```

2.0.9 Segment Tree

segtree.h

813343, 30 lines

```
template <class T, T (*op)(T, T), T (*e)()>
struct segtree {
  int n,size; V<T> d;
  segtree(const V<T>& v){
      n=sz(v),size=1;
      while(size<n)size<<=1;
      d=V<T>(2 * size, e());

      rep(i,0,n)d[size + i] = v[i];

      for(int i=size-1;i>=1;i--) update(i);
    }
    void update(int k) {
      d[k] = op(d[2 * k], d[2 * k + 1]);
    }
    void set(int p, T x) {
        assert(0 <= p && p < n);
        p += size;  d[p] = x;
        while(p > 1)update(p >>=1);
    }
    T prod(int l, int r) const { // [l,r)
      assert(0 <= l && l <= r && r <= n);
        T sml = e(), smr = e();
        for(l += size ,r += size ; l<r ;  l>>=1,r>>=1 )
            {
            if (l & 1) sml = op(sml, d[l++]);
            if (r & 1) smr = op(d[--r], smr);
        }
      return op(sml, smr);
    }
};
```

2.0.10 Lazy Segment Tree

lazy segtree.h

a1afbf, 123 lines

```
template <class S, S (*op)(S, S),S (*e)(),
          class F,  S (*Map)(F, S),
          F (*Com)(F, F),F (*id)()>
struct lazy_segtree {
    lazy_segtree(const V<S>& v) : _n(sz(v)) {
        size = 1 ;while(size<_n)size*=2 ;
        log = 31 - __builtin_clz(size);
        d =  V<S>(2 * size, e());
        lz = V<F>(size, id());
        rep(i ,0, _n ) d[size + i] = v[i];
        for (int i = size - 1; i >= 1; i--) update(i);
    }
    S prod(int l, int r) { // [l,r)
        assert(0 <= l && l <= r && r <= _n);
        if (l == r) return e();
        l += size; r += size;

        for (int i = log; i >= 1; i--) {
            if (((l >> i) << i) != l) push(l >> i);
            if (((r >> i) << i) != r) push((r - 1) >> i
                );
        }

        S sml = e(), smr = e();
        while (l < r) {
            if (l & 1) sml = op(sml, d[l++]);
            if (r & 1) smr = op(d[--r], smr);
            l >>= 1; r >>= 1;
        }

        return op(sml, smr);
    }
     void apply(int l, int r, F f) { // [l,r)
        assert(0 <= l && l <= r && r <= _n);
        if (l == r) return;

        l += size; r += size;

        for (int i = log; i >= 1; i--) {
            if (((l >> i) << i) != l) push(l >> i);
            if (((r >> i) << i) != r) push((r - 1) >> i
                );
        }

        {
            int l2 = l, r2 = r;
            while (l < r) {
                if (l & 1) all_apply(l++, f);
                if (r & 1) all_apply(--r, f);
                l >>= 1; r >>= 1;
            }
            l = l2; r = r2;
        }

        for (int i = 1; i <= log; i++) {
            if (((l >> i) << i) != l) update(l >> i);
            if (((r >> i) << i) != r) update((r - 1) >>
                 i);
        }
    }
    template <class G> int max_right(int l, G g) {
        assert(0 <= l && l <= _n);
        assert(g(e()));
        if (l == _n) return _n;
        l += size;
        for (int i = log; i >= 1; i--) push(l >> i);
```

```
        S sm = e();
        do {
            while (l % 2 == 0) l >>= 1;
            if (!g(op(sm, d[l]))) {
                while (l < size) {
                    push(l);
                    l = (2 * l);
                    if (g(op(sm, d[l]))) {
                        sm = op(sm, d[l]);
                        l++;
                    }
                }
                return l - size;
            }
            sm = op(sm, d[l]);
            l++;
        } while ((l & -l) != l);
        return _n;
    }
    template <class G> int min_left(int r, G g) {
        assert(0 <= r && r <= _n);
        assert(g(e()));
        if (r == 0) return 0;
        r += size;
        for (int i = log; i >= 1; i--) push((r - 1) >>
            i);
        S sm = e();
        do {
            r--;
            while (r > 1 && (r % 2)) r >>= 1;
            if (!g(op(d[r], sm))) {
                while (r < size) {
                    push(r);
                    r = (2 * r + 1);
                    if (g(op(d[r], sm))) {
                        sm = op(d[r], sm);
                        r--;
                    }
                }
                return r + 1 - size;
            }
            sm = op(d[r], sm);
        } while ((r & -r) != r);
        return 0;
    }

  private:
    int _n, size, log;
    V<S> d ; V<F> lz;

    void update(int k) { d[k] = op(d[2 * k], d[2 * k +
        1]); }
    void all_apply(int k, F f) {
        d[k] = Map(f, d[k]);
        if (k < size) lz[k] = Com(f, lz[k]);
    }
    void push(int k) {
        all_apply(2 * k, lz[k]);
        all_apply(2 * k + 1, lz[k]);
        lz[k] = id();
    }
};
```

2.0.11 Persistent Segment Tree

Psegtree.h

85ad76, 50 lines

```
struct node{
  int64_t sum ;
  node *lc; node *rc;
  node(){ rc=lc=NULL;sum =0 ;
  }
};
node* build(int L,int R){
  node * cur  = new node() ;
   if(L == R){
      cur->sum=a[L ];
     return  cur;
   }
   int m=(L+R)/2;
   cur->lc = build(L,m);
   cur->rc= build(m+1,R);
   cur->sum = cur->lc-> sum + cur->rc->sum ;
   return cur ;
}
ll quary (node *cur, int a,int b, int L,int R){

    if(a>R  || b< L )return 0ll ;
    if(L>= a && R<=b )return cur-> sum ;
   int m=(L+R)/2;
   ll p1 =quary(cur->lc ,a,b ,L,m);
   ll q1 =quary( cur->rc,a,b ,m+1,R);
  return p1+ q1 ;
}
node* update(node *cur ,int i,int val ,int L,int R){
   if(L==R ){
     node * ne = new node() ;
     ne ->sum= val ;
     return  ne ;
   }
   int m=(L+R)/2;
   if( i<= m ){
    node * ne  = new node() ;
    ne-> lc =update(cur->lc, i,val ,L,m) ;
    ne-> rc = cur-> rc ;
    ne-> sum = ne->lc-> sum + ne->rc->sum ;
    return ne ;
  }else{
    node * ne  = new node() ;
    ne-> rc =update(cur->rc, i,val ,m+1,R ) ;

    ne-> lc = cur-> lc ;
    ne-> sum = ne->lc-> sum + ne->rc->sum ;
     return ne ;
   }

}
```

2.0.12 Trie

Trie.h

900d83, 56 lines

```
struct Trie{
   const int B=30;
   struct node{
       node *nxt[2] ;
       int sz;
       node(){nxt[0]=nxt[1]=NULL ;sz=0 ;}
   }*root;

   Trie (){ root=new node(); }
   void insert(int x){
     node *cur= root ;
      cur->sz++ ;
      for(int i=B;i>=0;i--){
          int b=x>>i&1;
          if(cur->nxt[b]==NULL)cur->nxt[b]=new node() ,
              cur=cur->nxt[b];
          else cur=cur->nxt[b] ;
          cur->sz++;
      }
   }
   ll get_max(int x ){
      node *cur= root ;
       ll ans =0 ;
      for(int i=B;i>=0;i--){
          int b=x>>i&1;
          if(cur->nxt[!b] && cur->nxt[!b]->sz>0)cur=cur
              ->nxt[!b] ,ans+=(1<<i);
          else cur=cur->nxt[b] ;
      }
      return ans ;
   }
   int query(int x, int k) { // number of values s.t.
       val ^ x < k
    node* cur = root;
     int  ans = 0;
    for (int i = B ; i >= 0; i--) {
      if (cur == NULL) break;
      int b1 = x >> i & 1, b2 = k >> i & 1;
      if (b2 == 1) {
        if (cur -> nxt[b1]) ans += cur -> nxt[b1] -> sz
            ;
        cur = cur -> nxt[!b1];
      } else cur = cur -> nxt[b1];
    }
    return ans;
  }
  void remove(int val) {
    node* cur = root;
    cur -> sz--;
    for (int i = B; i >= 0; i--) {
        int b = val >> i & 1;
        cur = cur -> nxt[b];
        cur -> sz--;
    }
  }
```

```
  void del(node* cur) {
    for (int i=0;i<2;i++) if(cur->nxt[i])del(cur->nxt[i
        ]);
    delete(cur);
  }
};
```

2.0.13 Hash Map

HashMap.h

Description: Hash map with mostly the same API as unordered_map, but ~3x faster. Uses 1.5x memory. Initial capacity must be a power of 2 (if provided).

d77092, 7 lines

```
#include <bits/extc++.h>
// To use most bits rather than just the lowest ones:
struct chash { // large odd number for C
  const uint64_t C = ll(4e18 * acos(0)) | 71;
  ll operator()(ll x) const { return __builtin_bswap64(
      x*C); }
};
__gnu_pbds::gp_hash_table<ll,int,chash> h({},{},{},{},{
    1<<16});
```

2.0.14 Mo's Algorithm

Mo.h

Description: Mo quary 0 base time: O((N + Q)sqrt(N)) , block size is sqrt(n)

189dda, 46 lines

```
int n =0 , q=0;
  const int Block = 335 ;
   struct  MO{
       int l,r , id ;
       bool operator<(const MO &e )const {
         return  make_pair(l/Block , r)< make_pair(e.l/
             Block ,e.r) ;
       }
  };
    vector<MO> quary(q) ;
    // in quary
    auto add=[&] (int id){ };
    auto rem=[&](int id){ };
    auto clc=[&]()->int { return ; };

    sort(all(quary));
    int cl  = 0 ,cr=  -1 ;
    vi ans (q) ;
    for (int i=0;i<q;i++) {
      auto &[l,r , id ]  = quary[i];
        while (cl > l)add( --cl);
        while (cr < r)add(++cr );
        while (cl < l)rem(cl++ );
        while (cr > r)rem (cr-- );
        ans[id]= clc () ;
    }
/*

   Mo tree quary  make 2n tour vec
   for path u to v  if(in[u]> in[v] )swap(u,v)
   if  u == lca(u,v) then quary = in[u] to in [v ] ;
   else quary  out[u] to in[v] then add lca node
   all are 0 base
*/

   vi tour , in(n),out(n) ;
   auto dfs=[&](auto dfs,int v,int pa)-> void {
        in[v]=sz(tour);
        tour.pb(v) ;
        for(auto u: G[v]){
           if(u!= pa ){
             dfs(dfs,u,v) ;
           }
        }
       out[v]=sz(tour);
       tour.pb(v);
    }; dfs(dfs,0,-1);
```

Mathematics (3)

basis.h

d12181, 15 lines

```
vector<int > bit(32) ;
int Sz=0 ;
void Insert(int x){
   for(int i=30;i>=0 ;i--){

       if((x>>i& 1) == 0 )continue ;

       if(bit[i] )  x^=bit[i] ;
       else {
        bit[i]=x ;
        Sz++ ;
        break ;
     }
   }
}
```

Matrix.h

Description: Basic operations on square matrices.

Usage: `Matrix<int, 3> A;`
`A.d = {{{{1,2,3}}, {{4,5,6}}, {{7,8,9}}}};`
`array<int, 3> vec = {1,2,3};`
`vec = (A^N) * vec;`

6ab5db, 26 lines

```
template<class T, int N> struct Matrix {
  typedef Matrix M;
  array<array<T, N>, N> d{};
  M operator*(const M& m) const {
    M a;
    rep(i,0,N) rep(j,0,N)
      rep(k,0,N) a.d[i][j] += d[i][k]*m.d[k][j];
    return a;
  }
  array<T, N> operator*(const array<T, N>& vec) const {
    array<T, N> ret{};
    rep(i,0,N) rep(j,0,N) ret[i] += d[i][j] * vec[j];
    return ret;
  }
  M operator^(ll p) const {
    assert(p >= 0);
    M a, b(*this);
    rep(i,0,N) a.d[i][i] = 1;
    while (p) {
      if (p&1) a = a*b;
      b = b*b;
      p >>= 1;
    }
    return a;
  }
};
```

3.1 Trigonometry

$$\sin(v+w) = \sin v\cos w + \cos v\sin w$$
$$\cos(v+w) = \cos v\cos w - \sin v\sin w$$
$$\tan(v+w) = \frac{\tan v + \tan w}{1 - \tan v\tan w}$$
$$\sin v + \sin w = 2\sin\frac{v+w}{2}\cos\frac{v-w}{2}$$
$$\cos v + \cos w = 2\cos\frac{v+w}{2}\cos\frac{v-w}{2}$$

$$(V+W)\tan(\frac{v-w}{2}) = (V-W)\tan(\frac{v+w}{2})$$

V, W are sides opposite to angles v, w.
$a\cos x + b\sin x = r\cos(x-\phi)$
$a\sin x + b\cos x = r\sin(x+\phi)$
where $r = \sqrt{a^2+b^2}, \phi = \text{atan2}(b,a)$.

3.2 Geometry

3.2.1 Triangles

Side lengths: a, b, c

Semiperimeter: $p = \frac{a+b+c}{2}$

Area: $A = \sqrt{p(p-a)(p-b)(p-c)}$

Circumradius: $R = \frac{abc}{4A}$

Inradius: $r = \frac{A}{p}$

Length of median (divides triangle into two equal-area triangles): $m_a = \frac{1}{2}\sqrt{2b^2+2c^2-a^2}$

Length of bisector (divides angles in two): $s_a = \sqrt{bc\left[1-(a/(b+c))^2\right]}$

Law of sines, cosines & tangents:
$\frac{\sin\alpha}{a} = \frac{\sin\beta}{b} = \frac{\sin\gamma}{c} = \frac{1}{2R}$.....(1)

$$a^2 = b^2 + c^2 - 2bc\cos\alpha .....(2)$$
$$\frac{a+b}{a-b} = \frac{\tan((\alpha+\beta)/2)}{\tan((\alpha-\beta)/2)} .....(3)$$

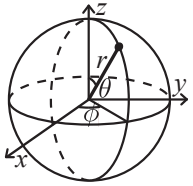
3.2.2 Quadrilaterals

With side lengths a, b, c, d, diagonals e, f, diagonals angle θ, area A and magic flux $F = b^2 + d^2 - a^2 - c^2$:

$$4A = 2ef \cdot \sin\theta = F\tan\theta = \sqrt{4e^2f^2 - F^2}$$

For cyclic quadrilaterals the sum of opposite angles is $180°$, $ef = ac + bd$, and $A = \sqrt{(p-a)(p-b)(p-c)(p-d)}$

3.2.3 Spherical coordinates



$$x = r\sin\theta\cos\phi \qquad r = \sqrt{x^2+y^2+z^2}$$
$$y = r\sin\theta\sin\phi \qquad \theta = \text{acos}(z/\sqrt{x^2+y^2+z^2})$$
$$z = r\cos\theta \qquad \phi = \text{atan2}(y, x)$$

3.3 Sums

$$c^a + c^{a+1} + \cdots + c^b = \frac{c^{b+1} - c^a}{c-1}, \quad c \neq 1$$

$$1^2 + 2^2 + 3^2 + \cdots + n^2 = \frac{n(2n+1)(n+1)}{6}$$
$$1^3 + 2^3 + 3^3 + \cdots + n^3 = \frac{n^2(n+1)^2}{4}$$
$$1^4 + 2^4 + 3^4 + \cdots + n^4 = \frac{n(n+1)(2n+1)(3n^2+3n-1)}{30}$$

$$1^4 + 2^4 + 3^4 + \cdots + n^4 = S_2 \times \frac{3n^2+3n-1}{5} = S_4$$

$$b\sum_{k=0}^{n-1}(a+kd)r^k = \frac{ab-(a+nd)br^n}{1-r} + \frac{dbr(1-r^n)}{(1-r)^2}$$

To compute $1^k + ... + n^k$ in $\mathcal{O}(k \lg k + k \lg MOD)$ compute first $t = k+2$ sums $y_1, ..., y_t$, then interpolate. Let $P = \prod_{i=1}^{t}(n-i)$. Then answer for n is

$$\sum_{i=1}^{t} \frac{P}{n-i} \cdot \frac{(-1)^{t-i}y_i}{(i-1)!(t-i)!}$$

Also $S_k = \frac{1}{k+1}\sum_{j=0}^{k}(-1)^j\binom{k+1}{j}B_j n^{k+1-j}$
where B_i are Bernoulli numbers.

3.4 Bitwise Formulas

Some properties of bitwise operations:

$$a|b = a \oplus b + a\&b$$
$$a \oplus (a\&b) = (a|b) \oplus b$$
$$b \oplus (a\&b) = (a|b) \oplus a$$
$$(a\&b) \oplus (a|b) = a \oplus b$$

Addition:

$$a + b = a|b + a\&b$$
$$a + b = a \oplus b + 2(a\&b)$$

Subtraction:

$$a - b = ((a\&b) \oplus ((a|b) \oplus a))$$
$$a - b = ((a|b) \oplus b) - ((a|b) \oplus a)$$
$$a - b = (a \oplus (a\&b)) - ((a|b) \oplus a)$$
$$a - b = (a \oplus (a\&b)) - (b \oplus (a\&b))$$
$$a - b = ((a|b) \oplus b) - (b \oplus (a\&b))$$

Working rules for logarithms:

1. $\log_b(mn) = \log_b m + \log_b n$
2. $\log_b\left(\frac{m}{n}\right) = \log_b m - \log_b n$
3. $\log_b(m^r) = r\log_b m$
4. $\log_b 1 = 0$
5. $\log_b b = 1$
6. $\log_b m = \frac{\log_a m}{\log_a b}$

3.5 Trivia

Pythagorean triples: The Pythagorean triples are uniquely generated by $a = k \cdot (m^2 - n^2)$, $b = k \cdot (2mn)$, $c = k \cdot (m^2 + n^2)$ with $m > n > 0$, $k > 0$, $\gcd(m, n) = 1$, both m, n not odd.
Primes: $p = 962592769$ is such that $2^{21} \mid p - 1$, which may be useful. For hashing use 970592641 (31-bit number), 31443539979727 (45-bit), 3006703054056749 (52-bit). There are 78498 primes less than 1 000 000.
Primitive roots modulo n exists iff $n = 1, 2, 4$ or, $n = p^k, 2p^k$ where p is an odd prime. Furthermore, the number of roots are $\phi(\phi(n))$.
To Find Generator g of M, factor $M-1$ and get the distinct primes p_i. If $g^{(M-1)/p_i} \neq 1(MODM)$ for each p_i then g is a valid root. Try all g until a hit is found (usually found very quick).
Esitmates: $\sum_{d|n} d = O(n \log\log n)$.
Prime count: 5133 upto 5e4. 9592 upto 1e5. 17984 upto 2e5. 78498 upto 1e6. 5761455 upto 1e8.
max NOD $\leq n$: 100 for $n = 5e4$. 500 for $n = 1e7$. 2000 for $n = 1e10$. 200 000 for $n = 1e19$.
max Unique Prime Factors: 6 upto 5e5. 7 upto 9e6. 8 upto 2e8. 9 upto 6e9. 11 upto 7e12. 15 upto 3e19.
Quadratic Residue: $(\frac{a}{p})$ is 0 if $p|a$, 1 if a is a quadratic residue, -1 otherwise. Euler: $(\frac{a}{p}) = a^{(p-1)/2}(\mod p)$ (prime). Jacobi: if $n = p_1^{e_1} \cdots p_k^{e_k}$ then $(\frac{a}{n}) = \prod(\frac{a}{p_i})^{e_i}$.
Chicken McNugget. If a, b coprime, there are $\frac{1}{2}(a-1)(b-1)$ numbers not of form $ax + by$ $(x, y \geq 0)$, the largest being $ab - a - b$.

Number Theory (4)

4.0.1 Modular Integer (Mint)

mint.h

e9effd, 15 lines

```
const int mod=1e9+7;
 struct mint{
  ll x;
  mint(ll x_=0):x(x_ % mod){if (x < 0) x += mod;}
  mint power(ll e) const {
      mint r = 1,b =*this;
      for (;e;e>>=1,b=b*b) if(e&1)r=r*b;
    return r;
  }
  mint inv(){return power(mod - 2);}
  mint operator+(mint b) { return mint(x + b.x); }
  mint operator-(mint b) { return mint(x - b.x); }
  mint operator*(mint b) { return mint(x * b.x); }
  mint operator/(mint b) { return *this * b.power(mod -
       2); }
 };
```

4.0.2 nCr

nCr.h

828473, 10 lines

```
V<mint> fct(1, 1), infct(1, 1);
 mint C(int n, int k) {
  if (k < 0 || k > n) return 0;

  while (sz(fct) < n + 1) {
    fct.pb(fct.back() * sz(fct) );
    infct.pb( mint(1) / fct.back());
  }
  return fct[n] * infct[k] * infct[n - k];
}
```

4.0.3 Euclid

euclid.h

Description: Finds two integers x and y, such that $ax+by = \gcd(a,b)$. If you just need gcd, use the built in __gcd instead. If a and b are coprime, then x is the inverse of $a \pmod b$.

33ba8f, 5 lines

```
ll euclid(ll a, ll b, ll &x, ll &y) {
  if (!b) return x = 1, y = 0, a;
  ll d = euclid(b, a % b, y, x);
  return y -= a/b * x, d;
}
```

4.0.4 Sieve of Eratosthenes

Eratosthenes.h

Description: Prime sieve for generating all primes up to a certain limit. isprime[i] is true iff i is a prime.
Time: lim=100'000'000 $\approx$ 0.8 s. Runs 30% faster if only odd indices are stored.

7c144c, 11 lines

```
const int MAX_PR = 5'000'000;
bitset<MAX_PR> isprime;
vi eratosthenesSieve(int lim) {
  isprime.set(); isprime[0] = isprime[1] = 0;
  for (int i = 4; i < lim; i += 2) isprime[i] = 0;
  for (int i = 3; i*i < lim; i += 2) if (isprime[i])
    for (int j = i*i; j < lim; j += i*2) isprime[j] =
        0;
  vi pr;
  rep(i,2,lim) if (isprime[i]) pr.push_back(i);
  return pr;
}
```

4.0.5 some functions

Let $n = p_1^{a_1} p_2^{a_2} \cdots p_k^{a_k}$.

$$\text{NOD: } \tau(n) = \prod_{i=1}^{k} (a_i + 1)$$

$$\text{SOD: } \sigma(n) = \prod_{i=1}^{k} \frac{p_i^{a_i+1} - 1}{p_i - 1}$$

$$\text{Euler: } \phi(n) = n \prod_{i=1}^{k} \left(1 - \frac{1}{p_i}\right)$$

4.0.6 Euler Phi Function

phi.h

Description: *Euler's* ϕ function is defined as $\phi(n) :=$ # of positive integers $\le n$ that are coprime with n. $\phi(1) = 1$, p prime $\Rightarrow \phi(p^k) = (p-1)p^{k-1}$, m, n coprime $\Rightarrow \phi(mn) = \phi(m)\phi(n)$. If $n = p_1^{k_1} p_2^{k_2} ... p_r^{k_r}$ then $\phi(n) = (p_1 - 1)p_1^{k_1-1} ... (p_r - 1)p_r^{k_r-1}$. $\phi(n) = n \cdot \prod_{p|n}(1 - 1/p)$. $\sum_{d|n} \phi(d) = n$, $\sum_{1 \le k \le n, \gcd(k,n)=1} k = n\phi(n)/2, n > 1$
Euler's thm: a, n coprime $\Rightarrow a^{\phi(n)} \equiv 1 \pmod n$.
Fermat's little thm: p prime $\Rightarrow a^{p-1} \equiv 1 \pmod p$ $\forall a$.

cf7d6d, 8 lines

```
const int LIM = 5000000;
int phi[LIM];

void calculatePhi() {
  rep(i,0,LIM) phi[i] = i&1 ? i : i/2;
  for (int i = 3; i < LIM; i += 2) if(phi[i] == i)
    for (int j = i; j < LIM; j += i) phi[j] -= phi[j] /
        i;
}
```

4.0.7 Sieve Factorization

Sievefactor.h

b710b2, 28 lines

```
int pclc=0;
vi least ,primes;
void RunSieve(int n) {
  least=vi(n + 1, 0);
  primes.clear();
  for (int i = 2; i <= n; i++) {
    if (least[i] == 0) {
      least[i] = i;
      primes.pb(i);
    }
    for (int x : primes) {
      if (x > least[i] || i * x > n) {
        break;
      }
      least[i * x] = x;
    }
  }
  pclc = n;
}
vi facList(int x) {
  assert(x<=pclc);
    vi res;
    while (x >1) {
      res.pb(least[x]);
      x /= least[x];
    }
    return res;
}
```

4.0.8 Linear Diophantine Equations

Lde.h

Description: Finds any integer solution (x_0, y_0) to the equation $a \cdot x + b \cdot y = c$, if it exists. Also computes $g = \gcd(a, b)$. All solutions can then be expressed as: $x = x_0 + k \cdot \frac{b}{g}$, $y = y_0 - k \cdot \frac{a}{g}$ for integer k.
Usage: int x, y, g;
if(find_any_solution(a, b, c, x, y, g)) {
x, y is one solution, g = gcd(a,b)
}
Memory: $\mathcal{O}(1)$
Time: $\mathcal{O}(log(max(a, b)))$

f55830, 10 lines

```
bool find_any_solution(int a, int b, int c, int &x0,
    int &y0, int &g) {
    g = euclid (abs(a), abs(b), x0, y0);
    if (c % g) return false;

    x0 *= c / g;
    y0 *= c / g;
    if (a < 0) x0 = -x0;
    if (b < 0) y0 = -y0;
    return true;
}
```

4.0.9 Chinese Remainder Theorem

crt.h

Description: Chinese Remainder Theorem.
crt(a, m, b, n) computes x such that $x \equiv a \pmod m$, $x \equiv b \pmod n$. If $|a| < m$ and $|b| < n$, x will obey $0 \le x < \text{lcm}(m, n)$. Assumes $mn < 2^{62}$.
Time: $\log(n)$

04d93a, 7 lines

```
ll crt(ll a, ll m, ll b, ll n) {
  if (n > m) swap(a, b), swap(m, n);
  ll x, y, g = euclid(m, n, x, y);
  assert((a - b) % g == 0); // else no solution
  x = (b - a) % n * x % n / g * m + a;
  return x < 0 ? x + m*n/g : x;
}
```

4.0.10 Mobius Function

mobius.h

Description: Count x in [L, R] with gcd(x, m) = k: k divides m sum over d dividing (m / k): mu[d] * (R / (k*d) - (L-1) / (k*d))
Count pairs (i, j) with gcd(i, j) = k: i in [l1, r1], j in [l2, r2] sum over d up to min(r1/k, r2/k): mu[d] * (r1 / (k*d) - (l1-1) / (k*d)) * (r2 / (k*d) - (l2-1) / (k*d))
Count pairs in array a with gcd(a[i], a[j]) = k: cnt[x] = number of elements divisible by x sum over d up to max(a)/k: mu[d] * cnt[d*k] * cnt[d*k]

All divisions are integer floor.

82c5a3, 11 lines
```
const int N = 5e5 + 9;
int mob[N];
void mobius() {
  mob[1] = 1;
  for (int i = 2; i < N; i++){
    mob[i]--;
    for (int j = i + i; j < N; j += i) {
      mob[j] -= mob[i];
    }
  }
}
```

4.0.11 Fast Factorization

fastfactor.h

Description: Pollard-rho randomized factorization algorithm. Returns prime factors of a number, in arbitrary order (e.g. 2299 -> {11, 19, 11}).
Time: $\mathcal{O}\left(n^{1/4}\right)$, less for numbers with small factors.

5abd1f, 43 lines
```
typedef unsigned long long ull;
ull modmul(ull a, ull b, ull M) {
  ll ret = a * b - M * ull(1.L / M * a * b);
  return ret + M * (ret < 0) - M * (ret >= (ll)M);
}
ull modpow(ull b, ull e, ull mod) {
  ull ans = 1;
  for (; e; b = modmul(b, b, mod), e /= 2)
    if (e & 1) ans = modmul(ans, b, mod);
  return ans;
}

bool isPrime(ull n) {
  if (n < 2 || n % 6 % 4 != 1) return (n | 1) == 3;
  ull A[] = {2, 325, 9375, 28178, 450775, 9780504,
      1795265022},
      s = __builtin_ctzll(n-1), d = n >> s;
  for (ull a : A) {   // ^ count trailing zeroes
    ull p = modpow(a%n, d, n), i = s;
    while (p != 1 && p != n - 1 && a % n && i--)
      p = modmul(p, p, n);
    if (p != n-1 && i != s) return 0;
  }
  return 1;
}

ull pollard(ull n) {
  ull x = 0, y = 0, t = 30, prd = 2, i = 1, q;
  auto f = [&](ull x) { return modmul(x, x, n) + i; };
  while (t++ % 40 || __gcd(prd, n) == 1) {
    if (x == y) x = ++i, y = f(x);
    if ((q = modmul(prd, max(x,y) - min(x,y), n))) prd
        = q;
    x = f(x), y = f(f(y));
  }
  return gcd(prd, n);
}
vector<ull> factor(ull n) {
  if (n == 1) return {};
  if (isPrime(n)) return {n};
  ull x = pollard(n);
  auto l = factor(x), r = factor(n / x);
  l.insert(l.end(), all(r));
  return l;
}
```

4.1 Big Integer Utilities

4.1.1 String Addition

stringAdd.cpp

Description: Adds two large integers represented as strings. Assumes both strings are of the same length. Returns the sum as a string.
Memory: $\mathcal{O}(n)$
Time: $\mathcal{O}(n)$ where n is the length of the strings

fc22e4, 22 lines
```
string add(string n1, string n2)
{
    string res = "";
    int sum, carry = 0;
    for (int i = sz(n1) - 1; i >= 0; i--)
    {
        sum = n1[i] - '0' + n2[i] - '0' + carry;
        if (sum > 9)
        {
            res = to_string(sum % 10) + res;
            carry = sum / 10;
        }
        else
        {
            res = to_string(sum) + res;
            carry = 0;
        }
    }
    if (carry)
        res = to_string(carry) + res;
    return res;
}
```

4.1.2 String Divisible Check

stringDivisible.cpp

Description: Checks if a large number represented as a string is divisible by an integer b. Returns the remainder; if remainder is 0, the number is divisible.
Memory: $\mathcal{O}(1)$
Time: $\mathcal{O}(n)$ where n is the length of the string

d06f67, 15 lines
```
int is_div(string a, int b)
{
    int j = 0;
    if (a[0] == '-')
        j = 1;
    if (b < 0)
        b = abs(b);
    ll rim = 0;
    for (; j < sz(a); j++)
    {
        rim = rim * 10 + (a[j] - '0');
        rim %= b;
    }
    return rim; // if rim=0 divisible else not
}
```

4.1.3 String Multiplication

stringMul.cpp

Description: Multiplies two large integers represented as strings and returns the product as a string.
Memory: $\mathcal{O}(n+m)$
Time: $\mathcal{O}(n*m)$ where n and m are lengths of n1 and n2

aad7b2, 20 lines
```
string mul(string n1, string n2)
{
  if (n1 == "0" || n2 == "0")
    return "0";
  string product(sz(n1) + sz(n2), 0);
  for (int i = sz(n1) - 1; i >= 0; i--)
  {
    for (int j = sz(n2) - 1; j >= 0; j--)
    {
      int n = (n1[i] - '0') * (n2[j] - '0') + product[i
          + j + 1];
      product[i + j + 1] = n % 10;
      product[i + j] += (n / 10);
    }
  }
  for (int i = 0; i < sz(product); i++)
  {
    product[i] += '0';
  }
  return product[0] == '0' ? product.substr(1) :
      product;
}
```

4.1.4 safin's NT temp

FactoriseSieve.cpp

Description: Generates primes up to M using a modified sieve (odd-only optimization). Also provides a factor() function that returns the prime factorization of n using the precomputed primes.
Usage: `sieve(1000000);`
`vector<int> f = factor(84); // returns {2,2,3,7}`
Memory: $\mathcal{O}(M)$
Time: Sieve: $\mathcal{O}(nloglogn)$, Factorization: $\mathcal{O}(primesuptosqrt(n))$

876ba8, 38 lines
```
const int M = 1000000; // max : 9e7
vector<int>primes;
bool marked[M];
bool isprime(int n) {
  if (n == 2)
    return true;
  if (n < 2 || n % 2 == 0)
    return false;
  return marked[n] == false;
}
void sieve(ll n) { // O(n log log n)
  for (ll i = 3; i * i <= n; i += 2) {
    if (marked[i] == false) // i is a prime
```

```
    {
      for (ll j = i * i; j <= n; j += i) {
        marked[j] = true;
      }
    }
  }
    for (int i = 0; i < n; i++) {
    if (isprime(i)) {
        primes.push_back(i);
    }
  }
}

vector < int > factor(int n) {
  vector < int > fact;
  for (int i = 0; primes[i] * primes[i] <= n; i++) {
    while (n % primes[i] == 0) {
      n /= primes[i];
      fact.pb(primes[i]);// cnt korbo and comment
    }
  }
  if (n > 1) fact.pb(n);
  return fact;
  // primes[i] is a , loop cnt is p, a^p
}
```

NODandSODsieve.cpp

Description: Precomputes Number of Divisors (NOD) and Sum of Divisors (SOD) for all numbers up to nmax using a modified sieve. Then prints tau(n) and sigma(n) for a given n.
Memory: $\mathcal{O}(N)$
Time: $\mathcal{O}(NlogN)$

23c0b4, 12 lines

```
const int nmax = 1e5 + 10;
int NOD[nmax];
int SOD[nmax];
void sieve(){ // O(NlogN)
    for(int i = 1; i<nmax; i++){
        for(int m = i; m < nmax; m += i){
            NOD[m]++;
            SOD[m] += i;
        }
    }

}
```

SPFandHPF.cpp

Description: Computes smallest prime factor (spf) for all numbers up to MAXN using a sieve. Allows quick factorization of any number <= MAXN.
Usage: `sieve(); // precompute spf`
`auto factors = getFactorization(100); // returns prime factors of 100`
Memory: $\mathcal{O}(MAXN)$
Time: $\mathcal{O}(MAXNloglogMAXN)$ for sieve, $\mathcal{O}(logn)$ per factorization

355ed3, 24 lines

```
int const MAXN = (int)1e7 + 5;
vector<int> spf(MAXN + 1, 1);
void sieve()
{
    spf[0] = 0;
    for (int i = 2; i <= MAXN; i++) {
        if (spf[i] == 1) {
            for (int j = i; j <= MAXN; j += i) {
                if (spf[j]== 1)
                    spf[j] = i;
            }
        }
    }
}

vector<int> getFactorization(int x) // with spf
{
    vector<int> ret;
    while (x != 1) {
        ret.push_back(spf[x]);
        x = x / spf[x];
    }
    return ret;
}
```

SegmentedSieve.cpp

Description: Generates all primes in (l, r) using a segmented sieve.
Usage: `auto primes1 = sieve(100); primes up to 100`
`auto primes2 = segmentedsieve(1000, 2000); primes in (1000 to 2000)`
Memory: $\mathcal{O}(r - l + sqrt(r))$
Time: sieve $\mathcal{O}(limitloglogglimit)$, segmentedsieve $\mathcal{O}((r - l)logr + sqrt(r))$

3dbf0f, 60 lines

```
// Generate all primes up to limit using sieve of
    eratosthenes
vector<int> sieve(int limit) {
    vector<bool> is_prime(limit + 1, true);
    is_prime[0] = is_prime[1] = false;
    for (int p = 2; p * p <= limit; ++p) {
        if (is_prime[p]) {
            for (int i = p * p; i <= limit; i += p) {
                is_prime[i] = false;
            }
        }
    }
    vector<int> primes;
    for (int p = 2; p <= limit; ++p) {
        if (is_prime[p]) {
            primes.push_back(p);
        }
    }
    return primes;
}

// Generate all primes from l to r using segmented
    sieve in O((r - l) log (r) + sqrt(r))
vector<ll> segmented_sieve(ll l, ll r) {
    if (l == 1) {
        l++;
    }
    int limit = sqrtl(r);
    while ((ll) limit * limit <= r) limit++;
    while ((ll) limit * limit > r) limit--;
    auto primes = sieve(limit);
    vector<bool> is_prime(r - l + 1, true);
    for (ll p : primes) {
        ll start = max((ll)p * p, (ll)(l + p - 1) / p *
            p);
        for (ll j = start; j <= r; j += p) {
            is_prime[j - l] = false;
        }
    }
    vector<ll> vec;
    for (ll i = l; i <= r; ++i) {
        if (is_prime[i - l]) {
            vec.push_back(i);
        }
    }
    return vec;
}

int main() {
    ios_base::sync_with_stdio(0);
    cin.tie(0);
    int t;
    cin >> t;
    while (t--) {
        ll l, r;
        cin >> l >> r;
        auto primes = segmented_sieve(l, r);
        for (auto p: primes) {
            cout << p << '\n';
        }
    }
    return 0;
}
```

bitsetSieve.cpp

Description: Provides a memory-efficient implementation of the Sieve of Eratosthenes using bitwise operations. Includes ‘sieve(n)‘ to generate all primes up to ‘n‘ and ‘isPrime(num)‘ to check primality of a number in O(1) time.
Memory: $\mathcal{O}(n/64)$
Time: $\mathcal{O}(nloglogn)$ for sieve, $\mathcal{O}(1)$ for isPrime

bcd863, 26 lines

```
#define M 100000000
int marked[M / 64 + 2];

#define on(x) (marked[x / 64] & (1 << ((x % 64) / 2)))
#define mark(x) marked[x / 64] |= (1 << ((x % 64) / 2))

void sieve(int n)
{
    for (int i = 3; i * i < n; i += 2)
    {
        if (!on(i))
        {
            for (int j = i * i; j <= n; j += i + i)
```

```
            {
                mark(j);
            }
        }
    }
}

bool isPrime(int num)
{
    return num > 1 && (num == 2 || ((num & 1) && !on(
        num)));
}

// memory efficient er jonno ei sieve
```

sieve upto 1e9.cpp

Memory: $\mathcal{O}\left(N(1/2) + blocksize + primecount\right)$
Time: Approximately 0.5s for N = 1e9

fe7395, 93 lines

```
// credit: min_25
// takes 0.5s for n = 1e9
vector<int> sieve(const int N, const int Q = 17, const
    int L = 1 << 15) {
  static const int rs[] = {1, 7, 11, 13, 17, 19, 23, 29
      };
  struct P {
    P(int p) : p(p) {}
    int p; int pos[8];
  };
  auto approx_prime_count = [] (const int N) -> int {
    return N > 60184 ? N / (log(N) - 1.1)
                     : max(1., N / (log(N) - 1.11)) +
                         1;
  };

  const int v = sqrt(N), vv = sqrt(v);
  vector<bool> isp(v + 1, true);
  for (int i = 2; i <= vv; ++i) if (isp[i]) {
    for (int j = i * i; j <= v; j += i) isp[j] = false;
  }

  const int rsize = approx_prime_count(N + 30);
  vector<int> primes = {2, 3, 5}; int psize = 3;
  primes.resize(rsize);

  vector<P> sprimes; size_t pbeg = 0;
  int prod = 1;
  for (int p = 7; p <= v; ++p) {
    if (!isp[p]) continue;
    if (p <= Q) prod *= p, ++pbeg, primes[psize++] = p;
    auto pp = P(p);
    for (int t = 0; t < 8; ++t) {
      int j = (p <= Q) ? p : p * p;
      while (j % 30 != rs[t]) j += p << 1;
      pp.pos[t] = j / 30;
    }
    sprimes.push_back(pp);
  }

  vector<unsigned char> pre(prod, 0xFF);
  for (size_t pi = 0; pi < pbeg; ++pi) {
    auto pp = sprimes[pi]; const int p = pp.p;
    for (int t = 0; t < 8; ++t) {
      const unsigned char m = ~(1 << t);
      for (int i = pp.pos[t]; i < prod; i += p) pre[i]
          &= m;
    }
  }

  const int block_size = (L + prod - 1) / prod * prod;
  vector<unsigned char> block(block_size); unsigned
      char* pblock = block.data();
  const int M = (N + 29) / 30;

  for (int beg = 0; beg < M; beg += block_size, pblock
      -= block_size) {
    int end = min(M, beg + block_size);
    for (int i = beg; i < end; i += prod) {
      copy(pre.begin(), pre.end(), pblock + i);
    }
    if (beg == 0) pblock[0] &= 0xFE;
    for (size_t pi = pbeg; pi < sprimes.size(); ++pi) {
      auto& pp = sprimes[pi];
      const int p = pp.p;
      for (int t = 0; t < 8; ++t) {
        int i = pp.pos[t]; const unsigned char m = ~(1
            << t);
        for (; i < end; i += p) pblock[i] &= m;
        pp.pos[t] = i;
      }
    }
    for (int i = beg; i < end; ++i) {
      for (int m = pblock[i]; m > 0; m &= m - 1) {
        primes[psize++] = i * 30 + rs[__builtin_ctz(m)
            ];
      }
    }
  }
  assert(psize <= rsize);
  while (psize > 0 && primes[psize - 1] > N) --psize;
  primes.resize(psize);
  return primes;
}

int32_t main() {
  ios_base::sync_with_stdio(0);
  cin.tie(0);
  int n, a, b; cin >> n >> a >> b; // n porjontw prime
      generate kore dibe
  auto primes = sieve(n);
  vector<int> ans;
  for (int i = b; i < primes.size() && primes[i] <= n;
      i += a) ans.push_back(primes[i]);
  // b theke shuru kore a ghor por por n porjontw prime
       gulo print korbe
  // input n=10 a=1 b=0 dile buja jabe
  cout << primes.size() << ' ' << ans.size() << '\n';
  for (auto x: ans) cout << x << ' '; cout << '\n';
  return 0;
}

// source link: https://github.com/ShahjalalShohag/code
    -library/blob/main/Number%20Theory/Sieve%20upto%201
    e9.cpp
//  https://judge.yosupo.jp/problem/enumerate_primes
```

mod bin exp.cpp

Description: this mod is bit slow, use $ans-= ans/mod * mod$
Memory: $\mathcal{O}(1)$
Time: $\mathcal{O}(logn)$ for bin exp and modInv, $\mathcal{O}(1)$ for other operations

39f9b5, 19 lines

```
ll bin_exp(ll a, ll n) //O(log(n))
{
  // a ^ n
  ll r = 1LL;
  int m = 1000000007;
  // Delete % m if we don't need it
  while (n) {
    if (n & 1) r = (r * a) % m, n--;
    else a = (a * a) % m, n /= 2LL;
  }
  return r;
}

const int mod2 = 1e9 + 7; // faster if const
inline ll MOD(ll a){ return (a % mod2 + mod2) % mod2; }
inline ll modAdd(ll a, ll b) { return MOD(MOD(a) + MOD(
    b)); }
inline ll modSub(ll a, ll b) { return MOD(MOD(a) - MOD(
    b)); }
inline ll modMul(ll a, ll b) { return MOD(MOD(a) * MOD(
    b)); }
inline ll modInv(ll a) { return bin_exp(a, mod2 - 2); }
```

4.2 More Number Theory

$$\sum_{k=1}^{n} \frac{1}{\gcd(k,n)} = \sum_{d|n} \frac{1}{d}\,\phi\left(\frac{n}{d}\right) = \frac{1}{n}\sum_{d|n} d\,\phi(d)$$

$$\sum_{k=1}^{n} \frac{k}{\gcd(k,n)} = \frac{n}{2}\cdot\frac{1}{n}\sum_{d|n} d\,\phi(d)$$

$$\sum_{k=1}^{n} \frac{n}{\gcd(k,n)} = 2\sum_{k=1}^{n} \frac{k}{\gcd(k,n)} - 1, \quad n > 1$$

$$\sum_{i=1}^{n}\sum_{j=1}^{n} [\gcd(i,j) = 1] = \sum_{d=1}^{n} \mu(d)\left\lfloor \frac{n}{d} \right\rfloor^2$$

$$\sum_{i=1}^{n}\sum_{j=1}^{n} \gcd(i,j) = \sum_{d=1}^{n} \phi(d)\left\lfloor \frac{n}{d} \right\rfloor^2$$

$$\sum_{i=1}^{n}\sum_{j=1}^{n} i\cdot j\,[\gcd(i,j)=1]=\sum_{i=1}^{n}\phi(i)\,i^2$$

$$\sum_{i=1}^{n}\sum_{j=1}^{n}\operatorname{lcm}(i,j)=\sum_{l=1}^{n}\left(\frac{(1+\lfloor n/l\rfloor)\lfloor n/l\rfloor}{2}\right)^2$$

4.2.1 Permutations

Factorial Values

n	1	2	3	4	5	6	7	8
$n!$	1	2	6	24	120	720	5040	40320

n	9	10	11	12	13
$n!$	362880	3628800	4.0e7	4.8e8	6.2e9

n	14	15	16	17	20	25
$n!$	8.7e10	1.3e12	2.1e13	3.6e14	2e18	2e25

4.3 Mobius Function

$$\mu(n)=\begin{cases}0 & n \text{ is not square free}\\ 1 & n \text{ has even number of prime factors}\\ -1 & n \text{ has odd number of prime factors}\end{cases}$$

Mobius Inversion:

$$g(n)=\sum_{d|n} f(d)\Leftrightarrow f(n)=\sum_{d|n}\mu(d)g(n/d)$$

Other useful formulas/forms:

$\sum_{d|n}\mu(d)=[n=1]$, $\phi(n)=\sum_{d|n}\mu(d)\frac{n}{d}$

$g(n)=\sum_{n|d} f(d)\Leftrightarrow f(n)=\sum_{n|d}\mu(\frac{d}{n})g(d)$

$g(n)=\sum_{1\le m\le n} f(\lfloor\frac{n}{m}\rfloor)\Leftrightarrow f(n)=\sum_{1\le m\le n}\mu(m)g(\lfloor\frac{n}{m}\rfloor)$

If f multiplicative, $\sum_{d|n}\mu(d)f(d)=\prod_{\text{prime } p|n}(1-f(p))$ and $\sum_{d|n}\mu^2(d)f(d)=\prod_{\text{prime } p|n}(1+f(p))$.

If $s_f(n)=\sum_{i=1}^{n} f(i)$ is a prefix sum of mulitplicative f then $s_{f*g}(n)=\sum_{1\le xy\le n} f(x)g(y)$. Then $s_f(n)=\{s_{f*g}(n)-\sum_{d=2}^{n} s_f(\lfloor n/d\rfloor)g(d)\}/g(1)$ where $f*g(n)=\sum_{d|n} f(d)g(n/d)$ (Dirichlet). Precompute (linear sieve) $O(n^{2/3})$ first values of s_f for complexity $O(n^{2/3})$.

Useful sums and convolutions: $\epsilon=\mu*\mathbf{1}$, $\mathrm{id}=\phi*\mathbf{1}$, $\mathrm{id}=g*\mathrm{id}_2$, where $\epsilon(n)=[n=1]$, $\mathbf{1}(n)=1$, $\mathrm{id}(n)=n$, $\mathrm{id}_k(n)=n^k$, $g(n)=\sum_{d|n}\mu(d)nd$.
coprime pairs in $[1,n]$ is $\sum_{d=1}^{n}\mu(d)\lfloor n/d\rfloor^2$. Sum of GCD pairs in $[1,n]$ is $\sum_{d=1}^{n}\phi(d)\lfloor n/d\rfloor^2$. Sum of LCM pairs in $[1,n]$ is $\sum_{d=1}^{n}(\frac{\lfloor n/d\rfloor(1+\lfloor n/d\rfloor)}{2})^2 g(d)$, where g is defined above with $g(p^k)=p^k-p^{k+1}$.

Graph (5)

5.0.1 Dijkstra

dijkstra.h

7b556c, 22 lines

```
vl dijkstra(const V<V<pll>> &g, int s) {
  int n =sz(g) ; assert(0 <= s && s < n);
  vl dist(n,ll(1e18));
  priority_queue<pll, V<pll>, greater<>> q;
  dist[s] = 0;
  q.emplace(dist[s], s);
  while (!q.empty()) {
    auto [expected,v] = q.top();
    q.pop();
    if (dist[v ] != expected ) continue;

      for(auto &[u,w]:g[v]){
            if(w+dist [v ]<dist[u]){
              dist[u]=w+dist[v ];
              q.emplace(dist[u],u);
            }
      }

  }
  return dist;
  // returns inf if there's no path
}
```

5.0.2 Bellman-Ford

BellmanFord.h

Description: Calculates shortest paths from s in a graph that might have negative edge weights. Unreachable nodes get dist = inf; nodes reachable through negative-weight cycles get dist = -inf. Assumes $V^2\max|w_i|<\sim 2^{63}$.
Time: $\mathcal{O}(VE)$

22de97, 26 lines

```
const ll inf = LLONG_MAX;
struct Ed {
  int a, b, w ;
  int s() { return a < b ? a : -a; }
};
struct Node { ll dist = inf; int prev = -1; };

void bellmanFord(vector<Node>& nodes, vector<Ed>& eds,
    int s) {
  nodes[s].dist = 0;
  sort(all(eds), [](Ed a, Ed b) { return a.s() < b.s();
       });

  int lim = sz(nodes) / 2 + 2; // /3+100 with shuffled
      vertices
  rep(i,0,lim) for (Ed ed : eds) {
    Node cur = nodes[ed.a], &dest = nodes[ed.b];
    if (abs(cur.dist) == inf) continue;
    ll d = cur.dist + ed.w;
    if (d < dest.dist) {
      dest.prev = ed.a;
      dest.dist = (i < lim-1 ? d : -inf);
    }
  }
  rep(i,0,lim) for (Ed e : eds) {
    if (nodes[e.a].dist == -inf)
      nodes[e.b].dist = -inf;
  }
}
```

5.0.3 Floyd-Warshall

Floyd Warshall.h

84a361, 15 lines

```
// inf means has no path
const ll inf = 2e18;
void Floyd_Warshall (V<vl> &dis){
  int n = sz(dis) ;
  rep(i,0,n )dis[i][i]=0;
   rep(k,0,n ){
     rep(i,0,n){
       rep(j,0,n){
          if(dis[i][k]!=inf && dis[k][j]!=inf ){
            dis[i][j]=min(dis[i][j],dis[i][k]+dis[k][j
                ]);
          }
       }
     }
  }
}
```

5.0.4 Strongly Connected Components

Scc.h

Description: usage : SCC(g,rg) g in garph and reverse graph ,return was has scc node with same val (0 base)

f130e0, 27 lines

```
pair<vi,vi> Fun (const V<vi> &g ,const vi &order  ){
     vi topo ,vis(sz(g)),was(sz(g))  ;
     int attempt=0;
     auto dfs=[&](auto dfs,int v)-> void {
             vis[v]=1;
             was[v]=attempt ;
            for(auto u: g[v]){
               if(vis[u]==0)
                  dfs(dfs, u);
            }
        topo.pb(v);
     };
     rep(i ,0,sz(order) ) {
           if(vis[order[i]] ==0 ){
                attempt++ ;
             dfs(dfs, order[i] ) ;
           }
     }
    reverse(all(topo))  ;
    return make_pair(topo,was) ;
}
vi SCC(const V<vi> &g ,V <vi>&rg){
    vi ord(sz(g)) ; iota(all(ord),0) ;
```

```
    auto [topo,_ ]= Fun(g, ord);
    auto [__,was]= Fun(rg,topo);
  return was;
}
```

5.0.5 Articulation Points and Bridges

artbri.h

Description: art and bri , bri has bridges (u,v) if u to v is a bridge and art [v] =1 if art is a art point (0 base)

d27c96, 40 lines

```
int n , timer;
V<vi> adj;
V<pii > bri ;
vi vis, tin,Sz, low ,art ;
void dfs(int v, int p = -1) {
    vis[v] = true;
    Sz[v]=1  ;
    tin[v] = low[v] = timer++;
    int child =0 ;
    bool pas = false;
    for (int to : adj[v]) {
        if (to == p && !pas) {
            pas = true;
            continue;
        }
        if (vis[to]) {
            low[v] = min(low[v], tin[to]);
        } else {
            dfs(to, v);
            Sz[v]+=Sz[to] ;
            low[v] = min(low[v], low[to]);
            if (low[to] >= tin[v] && p!=-1)art[v]=1;
            if (low[to] > tin[v])bri.pb({v, to});
             ++child;
        }
    }
    if(p == -1 && child > 1)art[v ] =1 ;
}

void art_bri() {

    timer = 0; bri.clear() ;
    Sz=vi(n,0) ;
    vis=vi(n, 0);art=vi(n,0);
    tin= vi (n, -1);low=vi(n, -1);
    rep (i,0,n) {
      if(!vis[i])dfs(i);
    }

}
```

5.0.6 Euler Walk

eulerWalk.h

Description: Eulerian undirected/directed path/cycle algorithm. Input should be a vector of (dest, global edge index), where for undirected graphs, forward/backward edges have the same index. Returns a list of nodes in the Eulerian path/cycle with src at both start and end, or empty list if no cycle/path exists. To get edge indices back, add .second to s and ret.

Time: $\mathcal{O}(V+E)$

780b64, 15 lines

```
vi eulerWalk(vector<vector<pii>>& gr, int nedges, int
    src=0) {
  int n = sz(gr);
  vi D(n), its(n), eu(nedges), ret, s = {src};
  D[src]++; // to allow Euler paths, not just cycles
  while (!s.empty()) {
    int x = s.back(), y, e, &it = its[x], end = sz(gr[x
        ]);
    if (it == end){ ret.push_back(x); s.pop_back();
        continue; }
    tie(y, e) = gr[x][it++];
    if (!eu[e]) {
      D[x]--, D[y]++;
      eu[e] = 1; s.push_back(y);
    }}
  for (int x : D) if (x < 0 || sz(ret) != nedges+1)
      return {};
  return {ret.rbegin(), ret.rend()};
}
```

5.0.7 Binary Lifting

BinaryLifting.h

Description: Calculate power of two jumps in a tree, to support fast upward jumps and LCAs. Assumes the root node points to itself.

Time: construction $\mathcal{O}(N \log N)$, queries $\mathcal{O}(\log N)$

bfce85, 25 lines

```
vector<vi> treeJump(vi& P){
  int on = 1, d = 1;
  while(on < sz(P)) on *= 2, d++;
  vector<vi> jmp(d, P);
  rep(i,1,d) rep(j,0,sz(P))
    jmp[i][j] = jmp[i-1][jmp[i-1][j]];
  return jmp;
}

int jmp(vector<vi>& tbl, int nod, int steps){
  rep(i,0,sz(tbl))
    if(steps&(1<<i)) nod = tbl[i][nod];
  return nod;
}

int lca(vector<vi>& tbl, vi& depth, int a, int b) {
  if (depth[a] < depth[b]) swap(a, b);
  a = jmp(tbl, a, depth[a] - depth[b]);
  if (a == b) return a;
  for (int i = sz(tbl); i--;) {
    int c = tbl[i][a], d = tbl[i][b];
    if (c != d) a = c, b = d;
  }
  return tbl[0][a];
}
```

5.0.8 Lowest Common Ancestor (LCA)

Lca.h

Description: Data structure for computing lowest common ancestors in a tree (with 0 as root). C should be an adjacency list of the tree, either directed or undirected.

Time: $\mathcal{O}(N \log N + Q)$

0f62fb, 21 lines

```
struct LCA {
  int T = 0;
  vi time, path, ret;
  RMQ<int> rmq;

  LCA(vector<vi>& C) : time(sz(C)), rmq((dfs(C,0,-1),
      ret)) {}
  void dfs(vector<vi>& C, int v, int par) {
    time[v] = T++;
    for (int y : C[v]) if (y != par) {
      path.push_back(v), ret.push_back(time[v]);
      dfs(C, y, v);
    }
  }

  int lca(int a, int b) {
    if (a == b) return a;
    tie(a, b) = minmax(time[a], time[b]);
    return path[rmq.query(a, b)];
  }
  //dist(a,b){return depth[a] + depth[b] - 2*depth[lca(
      a,b)];}
};
```

5.0.9 Heavy-Light Decomposition (HLD)

HLD.h

Description: Decomposes a tree into vertex disjoint heavy paths and light edges such that the path from any leaf to the root contains at most log(n) light edges. Code does additive modifications and max queries, but can support commutative segtree modifications/queries on paths and subtrees. Takes as input the full adjacency list. VALS_EDGES being true means that values are stored in the edges, as opposed to the nodes. All values initialized to the segtree default. Root must be 0. build ord[Id(i)] = a[i] for segtree querySubtree(u) return [l,r) Path(u,v) return vec of [l,r)

Time: $\mathcal{O}\left((\log N)^2\right)$

54f909, 41 lines

```
template <bool VALS_EDGES = false >
struct HLD {
  int N, tim = 0;
  vector<vi> adj;
  vi par, siz, rt, pos;

  HLD(vector<vi> adj_)
    : N(sz(adj_)), adj(adj_), par(N, -1), siz(N, 1),
      rt(N),pos(N) { dfsSz(0); dfsHld(0); }
  void dfsSz(int v) {
    for (int& u : adj[v]) {
```

```
      adj[u].erase(find(all(adj[u]), v));
      par[u] = v;
      dfsSz(u);
      siz[v] += siz[u];
      if (siz[u] > siz[adj[v][0]]) swap(u, adj[v][0]);
    }
  }
  void dfsHld(int v) {
    pos[v] = tim++;
    for (int u : adj[v]) {
      rt[u] = (u == adj[v][0] ? rt[v] : u);
      dfsHld(u);
    }
  }
  void process(int u, int v, V<pii> &op) {
    for (;; v = par[rt[v]]) {
      if (pos[u] > pos[v]) swap(u, v);
      if (rt[u] == rt[v]) break;
      op.pb({pos[rt[v]], pos[v] + 1});
    }
     op.pb({pos[u] + VALS_EDGES, pos[v] + 1});
  }
  V<pii> Path(int u, int v) {
   V<pii> op ; process(u, v,op); return op ;
  }
  int Id (int v){  return pos[v]; }
  pii querySubtree(int v) {
    return  {pos[v] + VALS_EDGES, pos[v] + siz[v]} ;
  }
};
```

Geometry (6)

6.1 Geometric Basics

Point.h

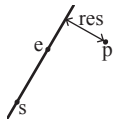
Description: Class to handle points in the plane. T can be e.g. double or long long. (Avoid int.)

ef0c0e, 29 lines

```
template <class T> int sgn(T x) { return (x > 0) - (x <
    0); }
template<class T>
struct Point {
typedef Point P;
T x, y;
explicit Point(T _x=0, T _y=0) : x(_x),y(_y){}
bool operator<(P p) const { return tie(x,y) < tie(p.x,p
    .y); }
bool operator==(P p) const { return tie(x,y)==tie(p.x,p
    .y); }
P operator+(P p) const{return P(x+p.x,y+p.y);}
P operator-(P p) const{return P(x-p.x,y-p.y);}
P operator*(T d) const { return P(x*d, y*d); }
P operator/(T d) const { return P(x/d, y/d); }
T dot(P p) const { return x*p.x + y*p.y; }
T cross(P p) const { return x*p.y - y*p.x; }
T cross(P a, P b) const { return (a-*this).cross(b-*
    this); }
T dist2() const { return x*x + y*y; }
double dist() const { return sqrt((double)dist2()); }
// angle to x-axis in interval [-pi, pi]
double angle() const { return atan2(y, x); }
// makes dist() = 1
P unit() const { return *this/dist(); }
// rotate by +90 degree
P perp() const { return P(-y, x); }
P normal() const { return perp().unit(); }
//rotate 'a' radians ccw around (0,0)
P rotate(double a) const { return P(x*cos(a)-y*sin(a),x
    *sin(a)+y*cos(a)); }
friend ostream& operator<<(ostream& os, P p) {
  return os<<"("<< p.x << "," << p.y << ")";}
};
```

lineDistance.h

Description:
Returns the signed distance between point p and the line containing points a and b. Positive value on left side and negative on right as seen from a towards b. a==b gives nan. P is supposed to be Point<T> or Point3D<T> where T is e.g. double or long long. It uses products in intermediate steps so watch out for overflow if using int or long long. Using Point3D will always give a non-negative distance. For Point3D, call .dist on the result of the cross product.



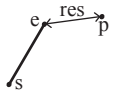
"Point.h" f6bf6b, 4 lines

```
template<class P>
double lineDist(const P& a, const P& b, const P& p) {
  return (double)(b-a).cross(p-a)/(b-a).dist();
}
```

SegmentDistance.h

Description:

Returns the shortest distance between point p and the line segment from point s to e.



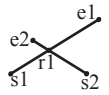
Usage: Point<double> a, b(2,2), p(1,1);
bool onSegment = segDist(a,b,p) < 1e-10;

"Point.h" 5c88f4, 6 lines

```
typedef Point<double> P;
double segDist(P& s, P& e, P& p) {
  if (s==e) return (p-s).dist();
  auto d = (e-s).dist2(), t = min(d,max(.0,(p-s).dot(e-
      s)));
  return ((p-s)*d-(e-s)*t).dist()/d;
}
```

SegmentIntersection.h

Description:
If a unique intersection point between the line segments going from s1 to e1 and from s2 to e2 exists then it is returned. If no intersection point exists an empty vector is returned. If infinitely many exist a vector with 2 elements is returned, containing the endpoints of the common line segment. The wrong position will be returned if P is Point<ll> and the intersection point does not have integer coordinates. Products of three coordinates are used in intermediate steps so watch out for overflow if using int or long long.



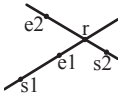
Usage: vector<P> inter = segInter(s1,e1,s2,e2);
if (sz(inter)==1)
cout << "segments intersect at " << inter[0] << endl;

"Point.h", "OnSegment.h" 9d57f2, 13 lines

```
template<class P> vector<P> segInter(P a, P b, P c, P d
    ) {
  auto oa = c.cross(d, a), ob = c.cross(d, b),
       oc = a.cross(b, c), od = a.cross(b, d);
  // Checks if intersection is single non-endpoint
      point.
  if (sgn(oa) * sgn(ob) < 0 && sgn(oc) * sgn(od) < 0)
    return {(a * ob - b * oa) / (ob - oa)};
  set<P> s;
  if (onSegment(c, d, a)) s.insert(a);
  if (onSegment(c, d, b)) s.insert(b);
  if (onSegment(a, b, c)) s.insert(c);
  if (onSegment(a, b, d)) s.insert(d);
  return {all(s)};
}
```

lineIntersection.h

Description:
If a unique intersection point of the lines going through s1,e1 and s2,e2 exists {1, point} is returned. If no intersection point exists {0, (0,0)} is returned and if infinitely many exists {-1, (0,0)} is returned. The wrong position will be returned if P is Point<ll> and the intersection point does not have integer coordinates. Products of three coordinates are used in intermediate steps so watch out for overflow if using int or ll.



Usage: auto res = lineInter(s1,e1,s2,e2);
if (res.first == 1)
cout << "intersection point at " << res.second << endl;

"Point.h" a01f81, 8 lines

```
template<class P>
pair<int, P> lineInter(P s1, P e1, P s2, P e2) {
  auto d = (e1 - s1).cross(e2 - s2);
  if (d == 0) // if parallel
    return {-(s1.cross(e1, s2) == 0), P(0, 0)};
  auto p = s2.cross(e1, e2), q = s2.cross(e2, s1);
  return {1, (s1 * p + e1 * q) / d};
}
```

sideOf.h

Description: Returns where p is as seen from s towards e. 1/0/-1 $\Leftrightarrow$ left/on line/right. If the optional argument *eps* is given 0 is returned if p is within distance *eps* from the line. P is supposed to be Point<T> where T is e.g. double or long long. It uses products in intermediate steps so watch out for overflow if using int or long long.

Usage: `bool left = sideOf(p1,p2,q)==1;`

"Point.h" 3af81c, 9 lines

```
template<class P>
int sideOf(P s, P e, P p) { return sgn(s.cross(e, p));
    }

template<class P>
int sideOf(const P& s, const P& e, const P& p, double
    eps) {
  auto a = (e-s).cross(p-s);
  double l = (e-s).dist()*eps;
  return (a > l) - (a < -l);
}
```

OnSegment.h

Description: Returns true iff p lies on the line segment from s to e. Use `(segDist(s,e,p)<=epsilon)` instead when using Point<double>.

"Point.h" c597e8, 3 lines

```
template<class P> bool onSegment(P s, P e, P p) {
  return p.cross(s, e) == 0 && (s - p).dot(e - p) <= 0;
}
```

LineProjectionReflection.h

Description: Projects point p onto line ab. Set refl=true to get reflection of point p across line ab instead. The wrong point will be returned if P is an integer point and the desired point doesn't have integer coordinates. Products of three coordinates are used in intermediate steps so watch out for overflow.

"Point.h" b5562d, 5 lines

```
template<class P>
P lineProj(P a, P b, P p, bool refl=false) {
  P v = b - a;
  return p - v.perp()*(1+refl)*v.cross(p-a)/v.dist2();
}
```

6.2 Circles

CircleIntersection.h

Description: Computes the pair of points at which two circles intersect. Returns false in case of no intersection.

"Point.h" 84d6d3, 11 lines

```
typedef Point<double> P;
bool circleInter(P a,P b,double r1,double r2,pair<P, P
    >* out) {
  if (a == b) { assert(r1 != r2); return false; }
  P vec = b - a;
  double d2 = vec.dist2(), sum = r1+r2, dif = r1-r2,
         p = (d2 + r1*r1 - r2*r2)/(d2*2), h2 = r1*r1 -
             p*p*d2;
  if (sum*sum < d2 || dif*dif > d2) return false;
  P mid = a + vec*p, per = vec.perp() * sqrt(fmax(0, h2
      ) / d2);
  *out = {mid + per, mid - per};
  return true;
}
```

CircleLine.h

Description: Finds the intersection between a circle and a line. Returns a vector of either 0, 1, or 2 intersection points. P is intended to be Point<double>.

"Point.h" e0cfba, 9 lines

```
template<class P>
vector<P> circleLine(P c, double r, P a, P b) {
  P ab = b - a, p = a + ab * (c-a).dot(ab) / ab.dist2()
      ;
  double s = a.cross(b, c), h2 = r*r - s*s / ab.dist2()
      ;
  if (h2 < 0) return {};
  if (h2 == 0) return {p};
  P h = ab.unit() * sqrt(h2);
  return {p - h, p + h};
}
```

CirclePolygonIntersection.h

Description: Returns the area of the intersection of a circle with a ccw polygon.

Time: $\mathcal{O}(n)$

"../../content/geometry/Point.h" a1ee63, 19 lines

```
typedef Point<double> P;
#define arg(p, q) atan2(p.cross(q), p.dot(q))
double circlePoly(P c, double r, vector<P> ps) {
  auto tri = [&](P p, P q) {
    auto r2 = r * r / 2;
    P d = q - p;
    auto a = d.dot(p)/d.dist2(), b = (p.dist2()-r*r)/d.
        dist2();
    auto det = a * a - b;
    if (det <= 0) return arg(p, q) * r2;
    auto s = max(0., -a-sqrt(det)), t = min(1., -a+sqrt
        (det));
    if (t < 0 || 1 <= s) return arg(p, q) * r2;
    P u = p + d * s, v = p + d * t;
    return arg(p,u) * r2 + u.cross(v)/2 + arg(v,q) * r2
        ;
  };
  auto sum = 0.0;
  rep(i,0,sz(ps))
    sum += tri(ps[i] - c, ps[(i + 1) % sz(ps)] - c);
  return sum;
}
```

6.3 Polygons

InsidePolygon.h

Description: Returns true if p lies within the polygon. If strict is true, it returns false for points on the boundary. The algorithm uses products in intermediate steps so watch out for overflow.

Usage: `vector<P> v = {P{4,4}, P{1,2}, P{2,1}};`
`bool in = inPolygon(v, P{3, 3}, false);`

Time: $\mathcal{O}(n)$

"Point.h", "OnSegment.h", "SegmentDistance.h" 2bf504, 11 lines

```
template<class P>
bool inPolygon(vector<P> &p, P a, bool strict = true) {
  int cnt = 0, n = sz(p);
  rep(i,0,n) {
    P q = p[(i + 1) % n];
    if (onSegment(p[i], q, a)) return !strict;
    //or: if (segDist(p[i], q, a) <= eps) return !
        strict;
    cnt ^= ((a.y<p[i].y) - (a.y<q.y)) * a.cross(p[i], q
        ) > 0;
  }
  return cnt;
}
```

PolygonArea.h

Description: Returns twice the signed area of a polygon. Clockwise enumeration gives negative area. Watch out for overflow if using int as T!

"Point.h" f12300, 6 lines

```
template<class T>
T polygonArea2(vector<Point<T>>& v) {
  T a = v.back().cross(v[0]);
  rep(i,0,sz(v)-1) a += v[i].cross(v[i+1]);
  return a;
}
```

PolygonCenter.h

Description: Returns the center of mass for a polygon.

Time: $\mathcal{O}(n)$

"Point.h" 9706dc, 9 lines

```
typedef Point<double> P;
P polygonCenter(const vector<P>& v) {
  P res(0, 0); double A = 0;
  for (int i = 0, j = sz(v) - 1; i < sz(v); j = i++) {
    res = res + (v[i] + v[j]) * v[j].cross(v[i]);
    A += v[j].cross(v[i]);
  }
  return res / A / 3;
}
```

ConvexHull.h

Description:

Returns a vector of the points of the convex hull in counter-clockwise order. Points on the edge of the hull between two other points are not considered part of the hull.

Time: $\mathcal{O}(n \log n)$

"Point.h" 310954, 13 lines

```
typedef Point<ll> P;
vector<P> convexHull(vector<P> pts) {
  if (sz(pts) <= 1) return pts;
  sort(all(pts));
  vector<P> h(sz(pts)+1);
```

```
  int s = 0, t = 0;
  for (int it = 2; it--; s = --t, reverse(all(pts)))
    for (P p : pts) {
      while (t >= s + 2 && h[t-2].cross(h[t-1], p) <=
          0) t--;
      h[t++] = p;
    }
  return {h.begin(), h.begin() + t - (t == 2 && h[0] ==
       h[1])};
}
```

HullDiameter.h

Description: Returns the two points with max distance on a convex hull (ccw, no duplicate/collinear points).
Time: $\mathcal{O}(n)$

"Point.h" c571b8, 12 lines

```
typedef Point<ll> P;
array<P, 2> hullDiameter(vector<P> S) {
  int n = sz(S), j = n < 2 ? 0 : 1;
  pair<ll, array<P, 2>> res({0, {S[0], S[0]}});
  rep(i,0,j)
    for (;; j = (j + 1) % n) {
      res = max(res, {(S[i] - S[j]).dist2(), {S[i], S[j
          ]}});
      if ((S[(j + 1) % n] - S[j]).cross(S[i + 1] - S[i
          ]) >= 0)
        break;
    }
  return res.second;
}
```

PointInsideHull.h

Description: Determine whether a point t lies inside a convex hull (CCW order, with no collinear points). Returns true if point lies within the hull. If strict is true, points on the boundary aren't included.
Time: $\mathcal{O}(\log N)$

"Point.h", "sideOf.h", "OnSegment.h" 71446b, 14 lines

```
typedef Point<ll> P;

bool inHull(const vector<P>& l, P p, bool strict = true
    ) {
  int a = 1, b = sz(l) - 1, r = !strict;
  if (sz(l) < 3) return r && onSegment(l[0], l.back(),
      p);
  if (sideOf(l[0], l[a], l[b]) > 0) swap(a, b);
  if (sideOf(l[0], l[a], p) >= r || sideOf(l[0], l[b],
      p)<= -r)
    return false;
  while (abs(a - b) > 1) {
    int c = (a + b) / 2;
    (sideOf(l[0], l[c], p) > 0 ? b : a) = c;
  }
  return sgn(l[a].cross(l[b], p)) < r;
}
```

LineHullIntersection.h

Description: Line-convex polygon intersection. The polygon must be ccw and have no collinear points. lineHull(line, poly) returns a pair describing the intersection of a line with the polygon: • $(-1,-1)$ if no collision, • $(i,-1)$ if touching the corner i, • (i,i) if along side $(i,i+1)$, • (i,j) if crossing sides $(i,i+1)$ and $(j,j+1)$. In the last case, if a corner i is crossed, this is treated as happening on side $(i,i+1)$. The points are returned in the same order as the line hits the polygon. `extrVertex` returns the point of a hull with the max projection onto a line.
Time: $\mathcal{O}(\log n)$

"Point.h" 7cf45b, 39 lines

```
#define cmp(i,j) sgn(dir.perp().cross(poly[(i)%n]-poly
    [(j)%n]))
#define extr(i) cmp(i + 1, i) >= 0 && cmp(i, i - 1 + n)
    < 0
template <class P> int extrVertex(vector<P>& poly, P
    dir) {
  int n = sz(poly), lo = 0, hi = n;
  if (extr(0)) return 0;
  while (lo + 1 < hi) {
    int m = (lo + hi) / 2;
    if (extr(m)) return m;
    int ls = cmp(lo + 1, lo), ms = cmp(m + 1, m);
    (ls < ms || (ls == ms && ls == cmp(lo, m)) ? hi :
        lo) = m;
  }
  return lo;
}

#define cmpL(i) sgn(a.cross(poly[i], b))
template <class P>
array<int, 2> lineHull(P a, P b, vector<P>& poly) {
  int endA = extrVertex(poly, (a - b).perp());
  int endB = extrVertex(poly, (b - a).perp());
  if (cmpL(endA) < 0 || cmpL(endB) > 0)
    return {-1, -1};
  array<int, 2> res;
  rep(i,0,2) {
    int lo = endB, hi = endA, n = sz(poly);
    while ((lo + 1) % n != hi) {
      int m = ((lo + hi + (lo < hi ? 0 : n)) / 2) % n;
      (cmpL(m) == cmpL(endB) ? lo : hi) = m;
    }
    res[i] = (lo + !cmpL(hi)) % n;
    swap(endA, endB);
  }
  if (res[0] == res[1]) return {res[0], -1};
  if (!cmpL(res[0]) && !cmpL(res[1]))
    switch ((res[0] - res[1] + sz(poly) + 1) % sz(poly)
        ) {
      case 0: return {res[0], res[0]};
      case 2: return {res[1], res[1]};
    }
  return res;
}
```

6.4 Misc. Point Set Problems

ClosestPair.h

Description: Finds the closest pair of points.
Time: $\mathcal{O}(n \log n)$

"Point.h" ac41a6, 17 lines

```
typedef Point<ll> P;
pair<P, P> closest(vector<P> v) {
  assert(sz(v) > 1);
  set<P> S;
  sort(all(v), [](P a, P b) { return a.y < b.y; });
  pair<ll, pair<P, P>> ret{LLONG_MAX, {P(), P()}};
  int j = 0;
  for (P p : v) {
    P d{1 + (ll)sqrt(ret.first), 0};
    while (v[j].y <= p.y - d.x) S.erase(v[j++]);
    auto lo = S.lower_bound(p - d), hi = S.upper_bound(
        p + d);
    for (; lo != hi; ++lo)
      ret = min(ret, {(*lo - p).dist2(), {*lo, p}});
    S.insert(p);
  }
  return ret.second;
}
```

6.5 3D

Strings (7)

7.0.1 KMP

KMP.h

Description: pi[x] computes the length of the longest prefix of s that ends at x, other than s[0...x] itself (abacaba -> 0010123). Can be used to find all occurrences of a string.
Time: $\mathcal{O}(n)$

d4375c, 16 lines

```
vi pi(const string& s) {
  vi p(sz(s));
  rep(i,1,sz(s)) {
    int g = p[i-1];
    while (g && s[i] != s[g]) g = p[g-1];
    p[i] = g + (s[i] == s[g]);
  }
  return p;
}

vi match(const string& s, const string& pat) {
  vi p = pi(pat + '\0' + s), res;
  rep(i,sz(p)-sz(s),sz(p))
    if (p[i] == sz(pat)) res.push_back(i - 2 * sz(pat))
        ;
  return res;
}
```

7.0.2 Z-function

Zfunc.h

Description: z[i] computes the length of the longest common prefix of s[i:] and s, except z[0] = 0. (abacaba -> 0010301)

Time: $\mathcal{O}(n)$

ee09e2, 12 lines

```
vi Z(const string& S) {
  vi z(sz(S));
  int l = -1, r = -1;
  rep(i,1,sz(S)) {
    z[i] = i >= r ? 0 : min(r - i, z[i - l]);
    while (i + z[i] < sz(S) && S[i + z[i]] == S[z[i]])
      z[i]++;
    if (i + z[i] > r)
      l = i, r = i + z[i];
  }
  return z;
}
```

7.0.3 Manacher's Algorithm

Manacher.h

Description: For each position in a string, computes p[0][i] = half length of longest even palindrome around pos i, p[1][i] = longest odd (half rounded down).

Time: $\mathcal{O}(N)$

e7ad79, 13 lines

```
array<vi, 2> manacher(const string& s) {
  int n = sz(s);
  array<vi,2> p = {vi(n+1), vi(n)};
  rep(z,0,2) for (int i=0,l=0,r=0; i < n; i++) {
    int t = r-i+!z;
    if (i<r) p[z][i] = min(t, p[z][l+t]);
    int L = i-p[z][i], R = i+p[z][i]-!z;
    while (L>=1 && R+1<n && s[L-1] == s[R+1])
      p[z][i]++, L--, R++;
    if (R>r) l=L, r=R;
  }
  return p;
}
```

7.0.4 Minimum Rotation

MinRotation.h

Description: Finds the lexicographically smallest rotation of a string.

Usage: `rotate(v.begin(), v.begin()+minRotation(v), v.end());`

Time: $\mathcal{O}(N)$

d07a42, 8 lines

```
int minRotation(string s) {
  int a=0, N=sz(s); s += s;
  rep(b,0,N) rep(k,0,N) {
    if (a+k == b || s[a+k] < s[b+k]) {b += max(0, k-1);
        break;}
    if (s[a+k] > s[b+k]) { a = b; break; }
  }
  return a;
}
```

7.0.5 Hashing

Hashing.h

Description: Static hashing for 0-indexed string. Intervals are $[l, r]$.

82983c, 20 lines

```
template<const ll M, const ll B>
struct Hashing {
  int n; V<ll> h, pw;
  Hashing(const string &s) : n(sz(s)),h(n+1),pw(n+1) {
    pw[0] = 1; // ^^ s is 0 indexed
    for (int i = 1; i <= n; ++i)
      pw[i] = (pw[i-1] * B) % M,
      h[i] = (h[i-1] * B + s[i-1]) % M;
  }
  ll eval(int l, int r) { // assert(l <= r);
    return (h[r+1] - ((h[l] * pw[r-l+1])%M) + M)%M;
} };
struct Double_Hash {
  using H1 = Hashing<916969619, 101>;
  using H2 = Hashing<285646799, 103>;
  H1 h1; H2 h2;
  Double_Hash(const string &s):h1(s),h2(s){}
  pii eval(int l, int r)
    { return {h1.eval(l,r), h2.eval(l,r)}; }
};
```

7.0.6 Dynamic Hashing

HashingDynamic.h

Description: Hashing with point updates on string (0-indexed). `upd(i, x):` `s[i] += x`. Intervals are $[l, r]$.

Time: $\mathcal{O}(n \log n)$

c51931, 33 lines

```
template<const ll M, const ll B>
struct Dynamic_Hashing {
  int n; V<ll> h, pw;
  void upd(int pos, int c_add) {
    if (c_add < 0) c_add = (c_add + M) % M;
    for (int i = ++pos; i <= n; i += i&-i)
      h[i] = (h[i]+c_add *1LL* pw[i - pos])%M;
  }
  ll get(int pos, int r = 0) {
    for (int i = ++pos, j = 0; i; i -= i&-i) {
      r = (r + h[i] * 1LL * pw[j]) % M;
      j += i&-i;
    } return r;
  }
  Dynamic_Hashing(const string &s) : n(sz(s)), h(n+1),
      pw(n+1) {
    pw[0] = 1; // ^^ s is 0 indexed
    for (int i = 1; i <= n; ++i) pw[i] = (pw[i-1] * 1LL
        * B) % M;
    for (int i = 0; i < n; ++i) upd(i, s[i]);
  }
  ll eval(int l, int r) { // assert(l <= r);
    return (get(r) - ((get(l-1) * 1LL * pw[r-l+1]) % M)
        + M) % M;
} };
struct Double_Dynamic {
  using DH1 = Dynamic_Hashing<916969619, 571>;
  using DH2 = Dynamic_Hashing<285646799, 953>;
  DH1 h1; DH2 h2;
  Double_Dynamic(const string &s) : h1(s), h2(s) {}
  void upd(int pos, int c_add) {
    h1.upd(pos, c_add);
    h2.upd(pos, c_add);
  } pll eval(int l, int r)
    { return {h1.eval(l,r), h2.eval(l,r)}; }
};
```

7.0.7 Suffix Array

suffixarray.h

Description: Builds suffix array for a string , `sa[i]` is the starting index of the suffix which is i'th in the sorted suffix array , The returned vector is of size n `lcp[i] = lcp(sa[i], sa[i+1])`, The input string must not contain any nul chars

Time: $\mathcal{O}(n \log n)$

f297aa, 66 lines

```
vi suffix_array(int n, const string&s, int char_bound=
    260) {
  vi a(n);
  if (n == 0)return a;
  if (char_bound != -1 ) {
    vi aux(char_bound, 0); int sum = 0;
    rep(i,0,n)aux[s[i]]++;
    rep(i,0,char_bound) {
      int add = aux[i];
      aux[i] = sum; sum += add;
    }
    rep( i,0,n) a[aux[s[i]]++] = i;
  } else {
    iota(all(a), 0);
    sort(all(a), [&s](int i, int j) { return s[i] < s[j
        ]; });
  }
  vi sbs(n),  ptg(n),neg(n), gp(n);
  gp[a[0]] = 0;
  rep (i,1,n) {
    gp[a[i]] = gp[a[i - 1]] + (!(s[a[i]] == s[a[i -
        1]]));
  }
  int cnt = gp[a[n - 1]] + 1 , step = 1;
  while (cnt < n) {
    int at = 0;
    rep (i,n - step,n )sbs[at++] = i;

    rep(i,0,n) {
      if (a[i] - step >= 0) {
        sbs[at++] = a[i] - step;
      }
    }
    for (int i = n - 1; i >= 0; i--)ptg[gp[a[i]]] = i;
    rep(i ,0,n) {
      int x = sbs[i];
      a[ptg[gp[x]]++] = x;
    }
    neg [a[0]] = 0;
    rep(i,1,n) {
```

```
      if (gp[a[i]] != gp[a[i - 1]]) {
        neg [a[i]] = neg[a[i - 1]] + 1;
      } else {
        int pre = (a[i - 1] + step >= n ? -1 : gp[a[i -
             1] + step]);
        int cur = (a[i] + step >= n ? -1 : gp[a[i] +
            step]);
        neg[a[i]] = neg[a[i - 1]] + (pre != cur);
      }
    }
    swap(gp, neg);
    cnt = gp[a[n - 1]] + 1;step <<= 1;
  } return a;
}

vi build_lcp(int n, const string &s, const vi &sa) {
  assert((int) sa.size() == n);
  vi pos(n), lcp(max(n - 1, 0)) ;
  rep (i,0,n)pos[sa[i]] = i; int k=0 ;
  rep(i,0,n) {
    k = max(k - 1, 0);
    if (pos[i] == n - 1)k = 0;
    else {
      int j = sa[pos[i] + 1];
      while (i + k < n && j + k < n && s[i + k] == s[j
          + k]) {
        k++;
      }
      lcp[pos[i]] = k;
    }
  } return lcp;
}
```

Basics and various (8)

8.1 Basics

8.1.1 Bitmask and Bitwise Operations

Bitmask and bitwise.cpp

Description: Utilities for bitmask enumeration and bit operations. Enumerating all subsets of k elements takes 2^k iterations. Bit operations use 0-based indexing:

Usage: `for (mask = 0; mask < (1 << k); mask++) iterate subsets.`

Time: $\mathcal{O}\left(k2^k\right)$ for enumeration, $\mathcal{O}(1)$ for each bit op.

c4c8d3, 35 lines

```
//getting all possible subset of k elements(k elements
    has 2^k subsets )
void bitmask(int k) //! O(k*(2^k))
{
  for (int j = 0; j < (1 << k); j++) {
    for (int i = 0; i < k; i++) {
      if (j & (1 << i)) {
        //* take the i-th element
      }
    }
  }
}

//bitwise basics 0 - based index
#define cnt_bits(x) __builtin_popcountll(x)
#define clz(x) __builtin_clzll(x)
#define ctz(x) __builtin_ctzll(x)

ll set0(ll n, ll k){
 n = (n | (1LL << k)) ^ (1LL << k);
    return n;
}
ll set1(ll n, ll k){
    n = (n | (1LL << k));
    return n;
}
ll togl(ll n, ll k){
    return (n ^ (1LL << k));
}
bool is_set(ll n, ll k){
    return (n & (1LL << k));
}

bool is_power_of_2(ll n){
  return (n>0 && !(n&(n-1)));
}
```

8.1.2 Compare Utility

compare.cpp

Time: $\mathcal{O}(1)$

1a3158, 18 lines

```
bool cmp(pair<int, int> A, pair<int, int> B) // pair
    compare
{
    if (A.first == B.first)
        return (A.second > B.second);
    //! if A.second is Larger than A will come before B
    return (A.first < B.first);
    //! if A.first is smaller than A will come before B
}

int double_comp(ld a, ld b) //compare between 2 double
{
    if (fabs(a - b) <= eps)
        return 0;
    return a < b ? -1 : 1;
    //! dbl cmp return 0 when a==b, -1 if a<b ,, 1 if a
        >b
}
```

8.1.3 Direction Array

Direction Array.cpp

Description: Direction vectors for grid and knight moves. - 4/8 directions $(dx[i], dy[i])$, $i = 0..7$: $(+1, 0)$ = Down, $(-1, 0)$ = Up, $(0, +1)$ = Right, $(0, -1)$ = Left, $(+1, +1)$ = Down-Right, $(+1, -1)$ = Down-Left, $(-1, +1)$ = Up-Right, $(-1, -1)$ = Up-Left - Knight moves $(kdx[i], kdy[i])$, $i = 0..7$

Time: $\mathcal{O}(1)$

f48822, 7 lines

```
// (dx, dy for grid-based problems) 4 directions and 4
    diagonal moves
const ll dx[] = {1, -1, 0, 0, 1, 1, -1, -1};
const ll dy[] = {0, 0, 1, -1, 1, -1, 1, -1};

// Knight's Move
ll kdx[] = {-2, -2, -1, -1,  1,  1,  2,  2};
ll kdy[] = {-1,  1, -2,  2, -2,  2, -1,  1};
```

8.1.4 Maximum Subarray Sum

Maximum subarray sum.cpp

Memory: $\mathcal{O}(N)$

Time: $\mathcal{O}(N)$

e61165, 9 lines

```
ll kadane_algo(const vector<ll> & v, ll n) {
  ll temp = 0, res = 0;
  for (ll i = 0; i < n; i++) {
    temp += v[i];
    if (temp < 0) temp = 0;
    res = max(res, temp);
  }
  return res;
}
```

8.2 Miscellaneous

8.2.1 Dates Utility

Dates.h

Description: Provides functions to convert between Gregorian calendar dates and Julian Day numbers. - 'dateToInt(y, m, d)': Converts a date (year, month, day) to its corresponding Julian Day number. - 'intToDate(jd, y, m, d)': Converts a Julian Day number back to a Gregorian date. - 'intToDay(jd)': Returns the day of the week (0 = monday, 5 = Saturday) from a Julian Day number.

Usage: `int jd = dateToInt(2025, 11, 14);`
`int y, m, d;`
`intToDate(jd, y, m, d);`
`int weekday = intToDay(jd);`

Memory: $\mathcal{O}(1)$ for all functions

Time: $\mathcal{O}(1)$ for all functions

892297, 20 lines

```
int intToDay(int jd) { return jd % 7; }
int dateToInt(int y, int m, int d) {
  return 1461 * (y + 4800 + (m - 14) / 12) / 4 +
    367 * (m - 2 - (m - 14) / 12 * 12) / 12 -
    3 * ((y + 4900 + (m - 14) / 12) / 100) / 4 +
    d - 32075;
}
void intToDate(int jd, int &y, int &m, int &d) {
  int x, n, i, j;
  x = jd + 68569;
  n = 4 * x / 146097;
  x -= (146097 * n + 3) / 4;
  i = (4000 * (x + 1)) / 1461001;
  x -= 1461 * i / 4 - 31;
  j = 80 * x / 2447;
```

```
  d = x - 2447 * j / 80;
  x = j / 11;
  m = j + 2 - 12 * x;
  y = 100 * (n - 49) + i + x;
}
```

8.2.2 Random Number Generator

Random.h

Description: Nice uniform real/int distribution wrapper

24d233, 9 lines

```
mt19937 rng(chrono::steady_clock::now().
    time_since_epoch().count());
// use mt19937_64 for long long
uniform_int_distribution<int> dist1(lo, hi);
uniform_real_distribution<> dist2(lo, hi);
// Usage
#define rand(l,r) uniform_int_distribution<ll>(l, r)(
    rng_64)
int val = rng(), val3 = dist1(rng);
ll val2 = rng_64(); double val4 = dist2(rng);
shuffle(vec.begin(), vec.end(), rng);
```

8.2.3 Longest Increasing Subsequence

LIS.h

Description: Compute indices for the longest increasing subsequence.

Time: $\mathcal{O}(N \log N)$

2932a0, 17 lines

```
template<class I> vi lis(const vector<I>& S) {
  if (S.empty()) return {};
  vi prev(sz(S));
  typedef pair<I, int> p;
  vector<p> res;
  rep(i,0,sz(S)) {
    // change 0 -> i for longest non-decreasing
        subsequence
    auto it = lower_bound(all(res), p{S[i], 0});
    if (it == res.end()) res.emplace_back(), it = res.
        end()-1;
    *it = {S[i], i};
    prev[i] = it == res.begin() ? 0 : (it-1)->second;
  }
  int L = sz(res), cur = res.back().second;
  vi ans(L);
  while (L--) ans[L] = cur, cur = prev[cur];
  return ans;
}
```

8.2.4 Fast Input

FastInput.h

Description: Read an integer from stdin. Usage requires your program to pipe in input from file.

Usage: `./a.out < input.txt`

Time: About 5x as fast as cin/scanf.

7b3c70, 15 lines

```
inline char gc() { // like getchar()
  static char buf[1 << 16];
  static size_t bc, be;
  if (bc >= be) {
    buf[0] = 0, bc = 0;
    be = fread(buf, 1, sizeof(buf), stdin);
  } return buf[bc++]; // returns 0 on EOF
}
int readInt() {
  int a, c;
  while ((a = gc()) < 40);
  if (a == '-') return -readInt();
  while ((c = gc()) >= 48) a = a*10+c-480;
  return a - 48;
}
```

8.2.5 Ternary Search

TernarySearch.h

Description: Find the smallest i in $[a, b]$ that maximizes $f(i)$, assuming that $f(a) < \ldots < f(i) \geq \cdots \geq f(b)$. To reverse which of the sides allows non-strict inequalities, change the $<$ marked with (A) to $<=$, and reverse the loop at (B). To minimize f, change it to $>$, also at (B).

Usage: `int ind = ternSearch(0,n-1,[&](int i){return a[i];});`

Time: $\mathcal{O}(\log(b-a))$

9155b4, 11 lines

```
template<class F>
int ternSearch(int a, int b, F f) {
  assert(a <= b);
  while (b - a >= 5) {
    int mid = (a + b) / 2;
    if (f(mid) < f(mid+1)) a = mid; // (A)
    else b = mid+1;
  }
  rep(i,a+1,b+1) if (f(a) < f(i)) a = i; // (B)
  return a;
}
```

troubleshoot.txt

52 lines

```
Pre-submit:
Write a few simple test cases if sample is not enough.
Are time limits close? If so, generate max cases.
Is the memory usage fine?
Could anything overflow?
Make sure to submit the right file.

Wrong answer:
Print your solution! Print debug output, as well.
Are you clearing all data structures between test cases
    ?
Can your algorithm handle the whole range of input?
Read the full problem statement again.
Do you handle all corner cases correctly?
Have you understood the problem correctly?
Any uninitialized variables?
Any overflows?
Confusing N and M, i and j, etc.?
Are you sure your algorithm works?
What special cases have you not thought of?
Are you sure the STL functions you use work as you
    think?
Add some assertions, maybe resubmit.
Create some testcases to run your algorithm on.
Go through the algorithm for a simple case.
Go through this list again.
Explain your algorithm to a teammate.
Ask the teammate to look at your code.
Go for a small walk, e.g. to the toilet.
Is your output format correct? (including whitespace)
Rewrite your solution from the start or let a teammate
    do it.

Runtime error:
Have you tested all corner cases locally?
Any uninitialized variables?
Are you reading or writing outside the range of any
    vector?
Any assertions that might fail?
Any possible division by 0? (mod 0 for example)
Any possible infinite recursion?
Invalidated pointers or iterators?
Are you using too much memory?
Debug with resubmits (e.g. remapped signals, see
    Various).

Time limit exceeded:
Do you have any possible infinite loops?
What is the complexity of your algorithm?
Are you copying a lot of unnecessary data? (References)
How big is the input and output? (consider scanf)
Avoid vector, map. (use arrays/unordered_map)
What do your teammates think about your algorithm?

Memory limit exceeded:
What is the max amount of memory your algorithm should
    need?
Are you clearing all data structures between test cases
    ?
```